

# Rent price prediction in Berlin

The goal of this project is to create the most suitable ML model to predict rent prices (Kaltmiete) in Berlin. Since Warmmiete (rent that includes other costs like electricity, heating, etc) isn't available in all listings, it was more suitable to use Kaltmiete instead. In this case, the best place to find the data for our dataset is real estate and apartment-search websites like Immowelt, WG-Gesucht, Immosuchmaschine, Kleineanzeigen, etc. For this project, I decided to use only Immowelt, since it included more data overall per listing, therefore more features can be included in the dataset. Also, it has a lot of new listings per day, so it is sufficient to build a proper dataset with enough samples if we gather data regularly.

## Data Cleaning & Preparation

### Part 1: Handling missing values

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [2]: df = pd.read_csv('../datasets/immowelt_listings.csv')
df.head(5)
```

Out[2]:

|   | listing_url                                                                                                       | cold_rent | warm_rent         | deposit | area                | room |
|---|-------------------------------------------------------------------------------------------------------------------|-----------|-------------------|---------|---------------------|------|
| 0 | <a href="https://www.immowelt.de/expose/f6e9dd76-dbe1-4...">https://www.immowelt.de/expose/f6e9dd76-dbe1-4...</a> | 1,499 €   | NaN               | 4500    | 58,3 m <sup>2</sup> | Zim  |
| 1 | <a href="https://www.immowelt.de/expose/56cf72dd-00ea-4...">https://www.immowelt.de/expose/56cf72dd-00ea-4...</a> | 1,178 €   | 1.350 € Warmmiete | 3534    | 45,7 m <sup>2</sup> | Zim  |
| 2 | <a href="https://www.immowelt.de/expose/591c45d0-01dd-4...">https://www.immowelt.de/expose/591c45d0-01dd-4...</a> | 5,130 €   | 6.130 € Warmmiete | 15390   | 270 m <sup>2</sup>  | Zim  |
| 3 | <a href="https://www.immowelt.de/expose/8e225383-b57c-4...">https://www.immowelt.de/expose/8e225383-b57c-4...</a> | 360 €     | 475 € Warmmiete   | 1080    | 93 m <sup>2</sup>   | Zim  |
| 4 | <a href="https://www.immowelt.de/expose/03a3d0b6-9fab-4...">https://www.immowelt.de/expose/03a3d0b6-9fab-4...</a> | 1,399 €   | 1.649 € Warmmiete | NaN     | 65,8 m <sup>2</sup> | Zim  |

5 rows × 27 columns



In [3]: `df.shape`

Out[3]: (22687, 27)

22687 samples and 27 features is a pretty good dataset

In [4]: `df.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22687 entries, 0 to 22686
Data columns (total 27 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   listing_url                          22687 non-null  object
1   cold_rent                            22289 non-null  object
2   warm_rent                           12409 non-null  object
3   deposit                             8311 non-null   object
4   area                                21656 non-null  object
5   rooms                               21679 non-null  object
6   floor                               18815 non-null  object
7   free_from                           13097 non-null  object
8   location                            21515 non-null  object
9   has_balkon                          10969 non-null  object
10  has_terrasse                        5991 non-null   object
11  has_garten                          2138 non-null   object
12  elevator                            9914 non-null   object
13  parking                             3460 non-null   object
14  keller                              10178 non-null  object
15  barrierefrei                        2649 non-null   object
16  einbaukueche                       12632 non-null  object
17  badewanne                           5288 non-null   object
18  dusche                              5612 non-null   object
19  flooring_type                       7795 non-null   object
20  energy_source                       9344 non-null   object
21  heating_type                        12388 non-null  object
22  property_condition                  9035 non-null   object
23  year_built                          11127 non-null  object
24  schufa_check                        22687 non-null  bool
25  number_of_images_posted             14269 non-null  float64
26  scraped_at                          22687 non-null  object
dtypes: bool(1), float64(1), object(25)
memory usage: 4.5+ MB

```

```
In [5]: df.isnull().sum()
```



```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22687 entries, 0 to 22686
Data columns (total 27 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   listing_url                          22687 non-null  object
1   cold_rent                            22289 non-null  object
2   warm_rent                            12409 non-null  object
3   deposit                              8311 non-null   object
4   area                                 21656 non-null  object
5   rooms                               21679 non-null  object
6   floor                               18815 non-null  object
7   free_from                            13097 non-null  object
8   location                             21515 non-null  object
9   has_balkon                           10969 non-null  object
10  has_terrasse                          5991 non-null   object
11  has_garten                           2138 non-null   object
12  elevator                             9914 non-null   object
13  parking                              3460 non-null   object
14  has_basement                         10178 non-null  object
15  is_barrier-free                       2649 non-null   object
16  has_built-in_kitchen                 12632 non-null  object
17  has_bathtub                          5288 non-null   object
18  has_shower                           5612 non-null   object
19  flooring_type                        7795 non-null   object
20  energy_source                        9344 non-null   object
21  heating_type                         12388 non-null  object
22  property_condition                   9035 non-null   object
23  year_built                           11127 non-null  object
24  schufa_check                         22687 non-null  bool
25  number_of_images_posted              14269 non-null  float64
26  scraped_at                           22687 non-null  object
dtypes: bool(1), float64(1), object(25)
memory usage: 4.5+ MB

```

We will assume that all the listings have advertised everything in their listing. Meaning if they haven't mentioned having an elevator in the advertisement, then the scraper has nothing to scrape and has returned null. For that we assume that the apartment does not have an elevator and replace 'null' with 'False'. We do this for all boolean features.

```

In [7]: cols = [
        'has_balkon', 'has_terrasse', 'has_garten', 'elevator', 'parking',
        'has_basement', 'is_barrier-free', 'has_built-in_kitchen',
        'has_bathtub', 'has_shower'
    ]

df[cols] = df[cols].astype("boolean").fillna(False)

```

```

In [8]: df.isnull().sum()

```

```
Out[8]: listing_url      0
        cold_rent      398
        warm_rent     10278
        deposit      14376
        area         1031
        rooms        1008
        floor        3872
        free_from     9590
        location      1172
        has_balkon      0
        has_terrasse     0
        has_garten      0
        elevator        0
        parking         0
        has_basement     0
        is_barrier-free  0
        has_built-in_kitchen  0
        has_bathtub      0
        has_shower       0
        flooring_type   14892
        energy_source   13343
        heating_type    10299
        property_condition 13652
        year_built     11560
        schufa_check      0
        number_of_images_posted 8418
        scraped_at       0
        dtype: int64
```

Now we are left with 14 features to clean. We will start with the features that are easy to fix, like 'number\_of\_images\_posted' and features that have very few missing areas, like 'cold\_rent'. For the 'cold\_rent', which is the predicted feature, luckily we only have 324 missing values, which is nothing compared to the number of samples our dataset has. We can either drop them or use an unsupervised learning algorithm to estimate those values.

```
In [9]: # For that we have to check the samples with the missing values to see if they are
        df[df.cold_rent.isnull()].sample(15)
```

Out[9]:

|              | listing_url                                                                                                       | cold_rent | warm_rent | deposit |
|--------------|-------------------------------------------------------------------------------------------------------------------|-----------|-----------|---------|
| <b>11545</b> | <a href="https://www.immowelt.de/expose/b705eeb5-929d-4...">https://www.immowelt.de/expose/b705eeb5-929d-4...</a> | NaN       | NaN       | NaN     |
| <b>12733</b> | <a href="https://www.immowelt.de/projekte/expose/k2vj93...">https://www.immowelt.de/projekte/expose/k2vj93...</a> | NaN       | NaN       | NaN     |
| <b>21286</b> | <a href="https://www.immowelt.de/expose/1d45bcb1-ebfe-4...">https://www.immowelt.de/expose/1d45bcb1-ebfe-4...</a> | NaN       | NaN       | NaN     |
| <b>4478</b>  | <a href="https://www.immowelt.de/projekte/expose/k2n6a3...">https://www.immowelt.de/projekte/expose/k2n6a3...</a> | NaN       | NaN       | NaN     |
| <b>6239</b>  | <a href="https://www.immowelt.de/expose/65ec240b-a726-4...">https://www.immowelt.de/expose/65ec240b-a726-4...</a> | NaN       | NaN       | NaN     |
| <b>72</b>    | <a href="https://www.immowelt.de/projekte/expose/k2ug93...">https://www.immowelt.de/projekte/expose/k2ug93...</a> | NaN       | NaN       | NaN     |
| <b>13434</b> | <a href="https://www.immowelt.de/expose/eeffb2b0-fba2-4...">https://www.immowelt.de/expose/eeffb2b0-fba2-4...</a> | NaN       | NaN       | NaN     |
| <b>11554</b> | <a href="https://www.immowelt.de/expose/be934ce2-a647-4...">https://www.immowelt.de/expose/be934ce2-a647-4...</a> | NaN       | NaN       | NaN     |
| <b>22665</b> | <a href="https://www.immowelt.de/expose/699ab277-c45f-4...">https://www.immowelt.de/expose/699ab277-c45f-4...</a> | NaN       | NaN       | NaN     |
| <b>11528</b> | <a href="https://www.immowelt.de/projekte/expose/k2mv83...">https://www.immowelt.de/projekte/expose/k2mv83...</a> | NaN       | NaN       | NaN     |
| <b>11537</b> | <a href="https://www.immowelt.de/expose/ee96cdb3-f97d-4...">https://www.immowelt.de/expose/ee96cdb3-f97d-4...</a> | NaN       | NaN       | NaN     |
| <b>20078</b> | <a href="https://www.immowelt.de/projekte/expose/k2n6a3...">https://www.immowelt.de/projekte/expose/k2n6a3...</a> | NaN       | NaN       | NaN     |
| <b>17962</b> | <a href="https://www.immowelt.de/expose/2e40e0ab-7fae-4...">https://www.immowelt.de/expose/2e40e0ab-7fae-4...</a> | NaN       | NaN       | NaN     |
| <b>2686</b>  | <a href="https://www.immowelt.de/expose/9ea2c75e-1764-4...">https://www.immowelt.de/expose/9ea2c75e-1764-4...</a> | NaN       | NaN       | NaN     |
| <b>6998</b>  | <a href="https://www.immowelt.de/projekte/expose/k2mv83...">https://www.immowelt.de/projekte/expose/k2mv83...</a> | NaN       | NaN       | NaN     |

15 rows × 27 columns

```
In [10]: with pd.option_context('display.max_colwidth', None):
          print(df[df.cold_rent.isnull()].sample(5)['listing_url'])
```

8347 https://www.immowelt.de/projekte/expose/k2mv832?serp\_view=list&search=distributionTypes%3DRent%26estateType%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D39#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_L

16610 https://www.immowelt.de/expose/ef47ce6c-b644-43e7-8208-cb16cfa83e17?serp\_view=list&search=distributionTypes%3DRent%26estateType%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D66#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_M

8967 https://www.immowelt.de/expose/76b9c9d3-1002-490f-8a0e-9a5065035113?serp\_view=list&search=distributionTypes%3DRent%26estateType%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D59#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_M

1751 https://www.immowelt.de/expose/35523e10-8c10-43ba-9742-97c67eccd32f?serp\_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location%3DAD08DE8634%26page%3D90&feature\_priceDropVariantB=true#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_L

11525 https://www.immowelt.de/projekte/expose/k2mv832?serp\_view=list&search=distributionTypes%3DRent%26estateType%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D40#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_L

Name: listing\_url, dtype: object

By checking the urls we notice that these links redirect to listings that don't belong to a specific apartment but are rather from agencies that advertise multiple other listings that we already scrapped, meaning these samples are invalid and should be dropped.

```
In [11]: old_rows = df.shape[0]
          df = df.dropna(subset=["cold_rent"])
          new_rows = df.shape[0]
          print(f"Rows removed: {old_rows - new_rows}")
```

Rows removed: 398

Now we decide how to handle samples with missing values in 'area' and 'rooms', which occur in nearly equal numbers. Unfortunately the samples that we dropped mostly had both these features, so probably we still have around 900 rows left. Before we decide on what to do with the samples, we have to first inspect them.

```
In [12]: df[df.area.isnull()].sample(5)
```



Out[12]:

|       | listing_url                                       | cold_rent | warm_rent            | deposit | area |
|-------|---------------------------------------------------|-----------|----------------------|---------|------|
| 17945 | https://www.immowelt.de/expose/1ce778a2-d6df-4... | 2949 €    | NaN                  | NaN     | NaN  |
| 12537 | https://www.immowelt.de/expose/1e27d31b-12ef-4... | 938 €     | NaN                  | NaN     | NaN  |
| 11433 | https://www.immowelt.de/expose/7f96a5c6-b35c-4... | 2680 €    | NaN                  | NaN     | NaN  |
| 3539  | https://www.immowelt.de/expose/5be177c8-1545-4... | 4238 €    | NaN                  | NaN     | NaN  |
| 17610 | https://www.immowelt.de/expose/26lirv2viibq?se... | 1998 €    | 2.377 €<br>Warmmiete | NaN     | NaN  |

5 rows × 27 columns



After sampling several different samples, these samples had mostly no features other than the prices, so they don't serve our model in any way, therefore we can safely drop them.

```
In [13]: df = df.dropna(subset=["area"])
```

```
In [14]: int(df.rooms.isnull().sum())
```

Out[14]: 161

```
In [15]: # Luckily, by dropping the samples with null area, the number of samples with null
df[df.rooms.isnull()].sample(5)
```

Out[15]:

|       | listing_url                                                                                                       | cold_rent | warm_rent                               | deposit | area       |
|-------|-------------------------------------------------------------------------------------------------------------------|-----------|-----------------------------------------|---------|------------|
| 16043 | <a href="https://www.immowelt.de/expose/dd3c1569-8b68-4...">https://www.immowelt.de/expose/dd3c1569-8b68-4...</a> | 676 €     | 676<br>€/Monat<br>Kaltmiete<br>zzgl. NK | 2028 €  | 30<br>m²   |
| 19817 | <a href="https://www.immowelt.de/expose/dbdddc68-f65c-4...">https://www.immowelt.de/expose/dbdddc68-f65c-4...</a> | 650 €     | 650<br>€/Monat<br>Warmmiete             | 650 €   | 20<br>m²   |
| 21252 | <a href="https://www.immowelt.de/expose/1eda4792-d7f8-4...">https://www.immowelt.de/expose/1eda4792-d7f8-4...</a> | 610 €     | 610<br>€/Monat<br>Kaltmiete<br>zzgl. NK | 1000 €  | 12,5<br>m² |
| 7462  | <a href="https://www.immowelt.de/expose/84034e76-5c0e-4...">https://www.immowelt.de/expose/84034e76-5c0e-4...</a> | 1350 €    | 1.650 €<br>Warmmiete                    | 4050 €  | 74<br>m²   |
| 4509  | <a href="https://www.immowelt.de/expose/ffe93561-3a62-4...">https://www.immowelt.de/expose/ffe93561-3a62-4...</a> | 1260 €    | NaN                                     | NaN     | 77<br>m²   |

5 rows × 27 columns



Interestingly this time the samples seems very legit, with most of them having all the features, even features like warm rent and deposit which are missing in many samples, therefore I don't want to drop them but rather find a reason why these values are missing in order to replace them and keep these samples. That's why I will inspect some listings first by sampling their urls.

```
In [16]: with pd.option_context('display.max_colwidth', None):  
          print(df[df.rooms.isnull()].sample(5)['listing_url'])
```

```

15517 https://www.immowelt.de/expose/840
34e76-5c0e-45a9-b309-55073b4b7b1f?serp_view=list&search=distributionTypes%3DRent%26e
stateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D11#ln=classified_se
arch_results&m=classified_search_results_classified_classified_detail_M
7317 https://www.immowelt.de/expose/3c6
d9071-5d20-4932-92b5-3e2fba1f01f0?serp_view=list&search=distributionTypes%3DRent%26e
stateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D5#ln=classified_sea
rch_results&m=classified_search_results_classified_classified_detail_XL
20394 https://www.immowelt.de/expose/dd3c1569-8b68-4194-9533-837d2664438a?serp_vi
ew=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location
s%3DAD08DE8634%26page%3D48&sort_order_ab_test=sort_order_reference#ln=classified_sea
rch_results&m=classified_search_results_classified_classified_detail_XL
9456 https://www.immowelt.de/expose/835
071d0-3025-435d-957c-52cfab8441fc?serp_view=list&search=distributionTypes%3DRent%26e
stateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D76#ln=classified_se
arch_results&m=classified_search_results_classified_classified_detail_M
4529 https://www.immowelt.de/expose/0ba
37606-3fb9-49c0-8db7-88234691a22f?serp_view=list&search=distributionTypes%3DRent%26e
stateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D37#ln=classified_se
arch_results&m=classified_search_results_classified_classified_detail_M
Name: listing_url, dtype: object

```

After inspecting several posts, I noticed that all the samples were valid and that these samples, with a missing number of rooms, were either WG apartments or simply just happened to be missing that value. Now we can either drop the values or come up with a way to estimate them. Now, to estimate, we can use features that impact the number of rooms, like area and number of images. We cannot use the price, that would cause data leakage.

```

In [17]: # Is it really worth it to do imputations? Let's check what part of the data set th
float(df.rooms.isnull().mean())*100

```

```

Out[17]: 0.751669078855222

```

```

In [18]: # 0.75%, it is basically irrelevant, we don't lose anything by dropping them.
# Doing imputations with these would add unnecessary noise
df = df.dropna(subset=["rooms"])

```

```

In [19]: # Let's recheck the remaning features with null values
df.isnull().sum()[df.isnull().sum() > 0]

```

```

Out[19]: warm_rent      9508
deposit      13102
floor        2753
free_from    8530
location     326
flooring_type 13698
energy_source 12139
heating_type  9063
property_condition 12411
year_built   10233
number_of_images_posted 7794
dtype: int64

```

Let's check the location since it has the fewest null values. Also, if you notice, the samples that don't have location have also dropped nearly the same as the number of samples that we dropped.

```
In [20]: df[df.location.isnull()].sample(5)
```

```
Out[20]:
```

|              | listing_url                                       | cold_rent | warm_rent            | deposit |            |
|--------------|---------------------------------------------------|-----------|----------------------|---------|------------|
| <b>17692</b> | https://www.immowelt.de/expose/0ff0f761-7e66-4... | 999 €     | 1.165 €<br>Warmmiete | NaN     | 3          |
| <b>12422</b> | https://www.immowelt.de/expose/8f5fbf7c-cf16-4... | 1750 €    | NaN                  | NaN     |            |
| <b>12741</b> | https://www.immowelt.de/expose/494c703b-4b74-4... | 636 €     | NaN                  | NaN     |            |
| <b>12739</b> | https://www.immowelt.de/expose/13531a58-926f-4... | 7175 €    | 8.175 €<br>Warmmiete | NaN     | 2          |
| <b>17770</b> | https://www.immowelt.de/expose/51bd788e-3911-4... | 2890 €    | NaN                  | NaN     | 4<br>Grunc |

5 rows × 27 columns



All the samples seem good, so there is no good reason to just drop them. Despite location being a very difficult feature to deal with, it is indeed a very powerful feature for our dataset. It is one of the main features to determine the rent of the apartment. I suggest, since it is a categorical feature, we replace the missing values with 'Unknown'. By doing so, even if we derive simple features from location like post number, street name and name of area, they are still gonna be categorical. Deriving a continuous feature like 'distance from city-center' or similar things is unlikely because we don't have the precise location. The model may find very useful patterns with these 'Unknown' values. Because, for example they might be missing it, the owner might not have put it for being a bad location and fearing it would lower the value of his property. We also have to consider the risk of this apartments not being in Berlin at all, without the location, we cannot ensure whether that is true or false. These apartments are most likely going to be in Berlin since that's where we scraped the data. But I want to ensure it first by sampling and checking some listings in order to find more information.

```
In [21]: with pd.option_context('display.max_colwidth', None):
          print(df[df.location.isnull()].sample(5)['listing_url'])
```

```

194      https://www.immowelt.de/expose/e6ba687a-e1ab-46b6-b66e-9db5bba2447f?serp_vie
ew=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locati
s%3DAD08DE8634%26page%3D168#ln=classified_search_results&m=classified_search_results
_classified_classified_detail_L
17594      https://www.immowelt.de/expose/87a5518c-8e2d-4f61-90c2-8f039b7ade4f?serp_vie
w=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locati
ons%3DAD08DE8634%26page%3D5#ln=classified_search_results&m=classified_search_results
_classified_classified_detail_L
12602      https://www.immowelt.de/expose/681f3b70-ab0b-495d-85dd-915a22a1ec7d?serp_vie
w=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locati
s%3DAD08DE8634%26page%3D43#ln=classified_search_results&m=classified_search_results_
classified_classified_detail_XL
17835      https://www.immowelt.de/expose/5c1b457d-24e9-4d57-94ab-d7b6409a9add?serp_vie
w=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locati
s%3DAD08DE8634%26page%3D36#ln=classified_search_results&m=classified_search_results_
classified_classified_detail_XL
12936      https://www.immowelt.de/expose/11c74691-d14e-42e3-aad2-6e17a2e
a44b0?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartme
nt%26locations%3DAD08DE8634#ln=classified_search_results&m=classified_search_results
_classified_classified_detail_L
Name: listing_url, dtype: object

```

By inspecting some samples, I noticed that the location was in Berlin. It was present in all the postings, but the reason why it is missing in our data set is either because they didn't put the feature but rather put it in the description, where our scraper can't get it, or our scraper simply didn't scrape it, even though it was present. Apparently, in some rare cases (less than 0.1%), the scraper failed to scrape it even though it was programmed to do it. Now the complexity of the decision-making increases because the data wasn't missing for the reason I thought. Even further, the locations were in average-priced locations, so not the worst places. If we replace the missing values with 'Unknown', we would encode scraper failure as a real-world category, which is wrong. If we drop the values, we lose almost 2% of our data set, and they are all meaningful data points. We postpone the decision on how to handle these samples until the remaining preprocessing steps are completed because the number of these samples may drop near 0.

```
In [22]: df[df.number_of_images_posted.isnull()].sample(5)
```

Out[22]:

|       | listing_url                                       | cold_rent | warm_rent            | deposit | area       |
|-------|---------------------------------------------------|-----------|----------------------|---------|------------|
| 22328 | https://www.immowelt.de/expose/01ce8884-917a-4... | 1222.84 € | 1.470 €<br>Warmmiete | NaN     | 63<br>m²   |
| 20736 | https://www.immowelt.de/expose/1f798696-501b-4... | 2100 €    | 2.530 €<br>Warmmiete | 6450 €  | 82,9<br>m² |
| 21509 | https://www.immowelt.de/expose/716b6cf9-004a-4... | 1415 €    | NaN                  | NaN     | 71<br>m²   |
| 21410 | https://www.immowelt.de/expose/ad83d1aa-1e66-4... | 1325 €    | 1.597 €<br>Warmmiete | NaN     | 71<br>m²   |
| 22270 | https://www.immowelt.de/expose/42f40998-dc45-4... | 1095 €    | NaN                  | 3600 €  | 55,2<br>m² |

5 rows × 27 columns



In [23]:

```
with pd.option_context('display.max_colwidth', None):
    print(df[df.number_of_images_posted.isnull()].sample(5)['listing_url'])
```

21127      https://www.immowelt.de/expose/33556830-03aa-4fc6-9a05-509b3114ea9a?serp\_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D73&sort\_order\_ab\_test=sort\_order\_reference#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_M

18640      https://www.immowelt.de/expose/265hu9qan3s3?serp\_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D48#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_M

16137      https://www.immowelt.de/expose/722030a4-0d5e-48e6-b5d5-cc9fa36fa01c?serp\_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D51#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_M

18043      https://www.immowelt.de/expose/8a74b5fe-90ba-46f8-81c3-88f098ecc058?serp\_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D9#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_XL

21047      https://www.immowelt.de/expose/c609e7cc-5ee7-48ed-bff6-a6ffc744619d?serp\_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D70&sort\_order\_ab\_test=sort\_order\_reference#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_M

Name: listing\_url, dtype: object

Before inspecting the samples with missing number of values, I was intuitively considering replacing NaN values with 0 thinking that the reason why they were NaN was because there were no images to scrape and therefore it didn't scrape anything. But almost all of the listings had images and for some reason it was just the scraper failing to scrape the text.

**Caution:** The scraper doesn't scrape the images themselves but rather the text saying

'there are x images' and from there it filters the number, but when there was no image, there was no text in the images and therefore no images.

```
In [24]: # Before we consider further, Let's check what values we have:
print(np.sort(df["number_of_images_posted"].unique()))
```

```
[ 3.  4.  5.  6.  7.  8.  9. 10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20.
 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38.
 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 50. 51. 52. 53. 55. 61. 67. 71.
 nan]
```

We have values like 1 and 2 missing, which is odd, especially considering the fact that in the samples, I just observed, there were many posts with 1 or 2 images. In this case dropping the rows is not an option, considering the fact that number\_of\_images has little to no impact on the price. Rather, it could be more useful to provide information about other features in case we need to do any imputations. In our data set we have many features and keeping a feature with 20% missing values and no way of imputing them is not a great idea, so it would be better to just drop it.

```
In [25]: df = df.drop(columns=["number_of_images_posted"])
```

```
In [26]: # floor => replace with unknown
df["floor"].unique()
# free-from => transform the immediately, Later, unknown
```

```
Out[26]: array(['5. Geschoss', '4. Geschoss', '3. Geschoss', '2. Geschoss',
                '1. Geschoss', '7. Geschoss', nan, 'EG', '11. Geschoss',
                '10. Geschoss', 'Geschoss 6/7', 'Geschoss -2/4', '6. Geschoss',
                'Geschoss 3/5', 'Geschoss 4/5', '4 Geschosse', '9. Geschoss',
                '8. Geschoss', '13. Geschoss', '17. Geschoss', 'Geschoss EG/3',
                '14. Geschoss', 'Geschoss -4/5', '15. Geschoss', 'Geschoss 4/6',
                'Geschoss 2/3', 'Geschoss 3/4', '12. Geschoss', 'Geschoss 15/21',
                'Geschoss 1/2', '20. Geschoss', 'Geschoss 2/4', 'Geschoss 2/2',
                'Geschoss EG/2', 'Geschoss EG/5', 'Geschoss 1/5', 'Geschoss EG/1',
                '5 Geschosse', 'Geschoss EG/6', '18. Geschoss', 'Geschoss 5/6',
                'Geschoss 2/5', '2 Geschosse', 'Geschoss 5/8', 'Geschoss -1/3',
                'Geschoss 1/8', '6 Geschosse', 'Geschoss 3/6', 'Geschoss 1/4',
                'Geschoss 2/6', 'Geschoss 3/3', 'Geschoss 1/6', '3 Geschosse',
                'Geschoss 4/7', 'Geschoss 1/3', 'Geschoss 3/9', 'Geschoss 6/6',
                'Geschoss 5/5', 'Geschoss -5/5', 'Geschoss 1/13', 'Geschoss EG/4',
                'Geschoss 5/7', '9 Geschosse', 'Geschoss 4/4', 'Geschoss -5/4',
                '7 Geschosse', 'Geschoss -1/5', 'Geschoss 1/7', 'Geschoss -2/3',
                'Geschoss 3/12', 'Geschoss 7/8', '16. Geschoss', 'Geschoss 2/10',
                'Geschoss EG/11', 'Geschoss 3/10', '21. Geschoss', 'Geschoss 7/7',
                'Geschoss 8/9', 'Geschoss 2/8', 'Geschoss 1/1', 'Geschoss -1/4',
                'Geschoss 9/11', 'Geschoss 10/20', 'Geschoss EG/8',
                'Geschoss EG/7', 'Geschoss 4/12', 'Geschoss 8/10'], dtype=object)
```

```
In [27]: with pd.option_context('display.max_colwidth', None):
print(df[df.floor.isnull()].sample(5)['listing_url'])
```

```

13570    https://www.immowelt.de/expose/735464d5-8f2c-40b7-9266-9d0e73de9974?serp_vie
ew=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location
s%3DAD08DE8634%26page%3D64#ln=classified_search_results&m=classified_search_results_
classified_classified_detail_M
16045    https://www.immowelt.de/expose/46b2734d-154a-46b9-8526-8a8e09930ecb?serp_vie
ew=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location
s%3DAD08DE8634%26page%3D47#ln=classified_search_results&m=classified_search_results_
classified_classified_detail_M
10490    https://www.immowelt.de/expose/8c85cad9-bf25-4506-901a-123031150cc2?serp_vie
ew=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location
s%3DAD08DE8634%26page%3D79#ln=classified_search_results&m=classified_search_results_
classified_classified_detail_L
14290    https://www.immowelt.de/expose/1f3d4869-5911-4166-b824-8370cfbc4812?serp_vie
ew=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location
s%3DAD08DE8634%26page%3D23#ln=classified_search_results&m=classified_search_results_
classified_classified_detail_M
7784     https://www.immowelt.de/expose/4313cddd-6aff-4c94-b52e-cb3fdeb3db3f?serp_vie
ew=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location
s%3DAD08DE8634%26page%3D20#ln=classified_search_results&m=classified_search_results_
classified_classified_detail_M
Name: listing_url, dtype: object

```

```

In [28]: with pd.option_context('display.max_colwidth', None):
          print(df[df.floor == '9 Geschosse'].sample(5)['listing_url'])

```

```

21961    https://www.immowelt.de/expose/307c62c5-40fb-4314-9542-a6f3f12d839e?serp_vie
ew=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location
s%3DAD08DE8634%26page%3D49&sort_order_ab_test=sort_order_reference#ln=classified_sea
rch_results&m=classified_search_results_classified_classified_detail_L
3727     https://www.immowelt.de/expose/307c62c5-40fb-4314-9542-a6f3f12d839e?serp_vie
w=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D23&feature_priceDropVariantB=true#ln=classified_sea
rch_results&m=classified_search_results_classified_classified_detail_L
6944     https://www.immowelt.de/expose/307c62c5-40fb-4314-9542-a6f3f12d839e?serp_vie
w=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D33#ln=classified_sea
rch_results&m=classified_search_results_classified_classified_detail_L
20438    https://www.immowelt.de/expose/307c62c5-40fb-4314-9542-a6f3f12d839e?serp_vie
w=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location
s%3DAD08DE8634%26page%3D50&sort_order_ab_test=sort_order_reference#ln=classified_sea
rch_results&m=classified_search_results_classified_classified_detail_L
20510    https://www.immowelt.de/expose/307c62c5-40fb-4314-9542-a6f3f12d839e?serp_vie
w=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location
s%3DAD08DE8634%26page%3D52&sort_order_ab_test=sort_order_reference#ln=classified_sea
rch_results&m=classified_search_results_classified_classified_detail_L
Name: listing_url, dtype: object

```

```

In [29]: with pd.option_context('display.max_colwidth', None):
          print(df[df.floor == 'Geschoss -2/3'].sample(5)['listing_url'])

```



```

16434 https://www.immowelt.de/expose/26fknlg5hddf?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D61#ln=classified_search_results&m=classified_search_results_classified_classified_detail_M
4592 https://www.immowelt.de/expose/26fknlg5hddf?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D35#ln=classified_search_results&m=classified_search_results_classified_classified_detail_M
21074 https://www.immowelt.de/expose/26fknlg5hddf?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D71&sort_order_ab_test=sort_order_reference#ln=classified_search_results&m=classified_search_results_classified_classified_detail_M
14623 https://www.immowelt.de/expose/26fknlg5hddf?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D66#ln=classified_search_results&m=classified_search_results_classified_classified_detail_M
13558 https://www.immowelt.de/expose/26fknlg5hddf?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D64#ln=classified_search_results&m=classified_search_results_classified_classified_detail_M
Name: listing_url, dtype: object

```

```

In [30]: with pd.option_context('display.max_colwidth', None):
          print(df[df.floor == 'Geschoss -1/3'].sample(2)['listing_url'])

```

```

3014 https://www.immowelt.de/expose/246a1kdzqnk?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D45&feature_priceDropVariantB=true#ln=classified_search_results&m=classified_search_results_classified_classified_detail_M
8098 https://www.immowelt.de/expose/261flizrfgkq?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D30#ln=classified_search_results&m=classified_search_results_classified_classified_detail_M
Name: listing_url, dtype: object

```

**After inspecting the values and some listings, we reach the conclusion that we have 6 types of values:**

1. Good values like '5. Geschoss' from which we can simply extract the floor number
2. Other good values like 'EG' just stands for 'ground floor' meaning floor number 0
3. Bad values that just state the number of total floors '9 Geschosse', but not in which floor the apartment is, we cannot conclude any information from it, like the apartment being on the first, ground floor or the highest floor because by inspecting some listings, we noticed that there simply wasn't a particular reason why the specific floor wasn't mentioned. These values are to be considered as null values.
4. Good values in a fraction format shaped => these values contain the number of the floor and the number of the total floors. We are only going to extract the number of the floor from these values.
5. Confusing negative fraction values => I called them confusing because some of them really meant negative floors like -1, but values like -2, -3, -4 and -5 meant actual floor number 2, 3 and so on rather than apartments buried into the ground.
6. NaN values

The best solution in this case is to get the floor number from each feature and leave them as nan values as they are, since we want to have a continuous feature rather than replacing it with 'Unknown' and converting the feature to a categorical one just for one type of value.

For the tricky negative-value values, we only leave -1 and convert the rest as positive.

```
In [31]: # We will write a custom function to do the formatting

import re

def format_floor(value):
    # Case 0: already numeric -> we don't do anything
    if isinstance(value, (int, float)) and not pd.isna(value):
        return int(value)

    # Case 1: NaN
    if pd.isna(value):
        return np.nan

    if not isinstance(value, str):
        return np.nan

    value = value.strip()

    # Case 2: Ground floor
    if value == "EG":
        return 0

    # Case 3: Simple format Like "5. Geschoss"
    match = re.match(r"(-?\d+)\.\s*Geschoss", value)
    if match:
        floor = int(match.group(1))
        return floor if floor == -1 else abs(floor)

    # Case 4: Fraction format Like "Geschoss 3/5" or "Geschoss -2/4"
    match = re.match(r"Geschoss\s+(-?\d+)\s*/\s*(\d+)", value)
    if match:
        floor = int(match.group(1))
        return floor if floor == -1 else abs(floor)

    # Case 5: Only building height Like "9 Geschosse"
    if re.match(r"\d+\s+Geschosse", value):
        return np.nan

    # Anything else: unknown / unusable
    return np.nan
```

```
In [32]: df["floor"] = np.sort(df["floor"].apply(format_floor).astype("Int64"))
df["floor"].unique()
```

```
Out[32]: array([-1.,  0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11.,
        12., 13., 14., 15., 16., 17., 18., 20., 21., nan])
```

```
In [33]: df.isnull().sum()
```

```
Out[33]: listing_url      0
        cold_rent        0
        warm_rent      9508
        deposit       13102
        area            0
        rooms           0
        floor          3122
        free_from       8530
        location        326
        has_balkon      0
        has_terrasse     0
        has_garten      0
        elevator        0
        parking         0
        has_basement     0
        is_barrier-free  0
        has_built-in_kitchen 0
        has_bathtub     0
        has_shower      0
        flooring_type   13698
        energy_source   12139
        heating_type    9063
        property_condition 12411
        year_built     10233
        schufa_check     0
        scraped_at      0
        dtype: int64
```

Features like warm\_rent and deposit are directly linked with cold\_rent - the target variable.

warm\_rent = cold\_rent + utilities

deposit is often a direct multiple of rent (e.g. 2-3×)

Using them to train a model would cause data leakage, therefore we don't need them to train our model. We have to drop them.

```
In [34]: df = df.drop(columns=["warm_rent", "deposit"])
```

```
In [35]: # Let's check what values does 'free_from' has:
df["free_from"].unique()
```

```
Out[35]: array(['15.02.2026', '01.03.2026', 'sofort', '29.12.2025', '01.02.2026',
               '01.01.2026', 'ab sofort', nan, 'nach absprache', 'nach wunsch',
               '15.01.2026', '31.12.2025', '10.01.2026', 'nach vereinbarung',
               'ab 01.02.2026', '08.01.2026', '28.02.2026', '1.06.2025',
               'herbst 2025', 'auf anfrage', '14.02.2026', 'ab 01.04.2026',
               '13.02.2026', 'neuwertig', '12.01.2026', '01.04.2026',
               '16.01.2026', '31.01.2026', 'jan 2026', '29.01.2026', '02.02.2026',
               'mitte januar', '09.02.2026', '23.02.2026', '12.02.2026',
               '21.01.2026', 'ab märz 2026', 'ab 01.10.2025', '23.01.2026',
               '10.02.2026', 'ab 01.11.2025', '11.01.2026', '18.01.2026',
               '01.05.2026', 'bezugsfrei', '16.02.2026', '08.02.2026',
               '14.01.2026', '15.01.2026 sofort', '2013', '30.01.2026',
               '05.02.2026', '20.02.2026', '20.01.2026', '19.01.2026',
               '15.03.2026', '26.01.2026', '13.04.2026', '03.03.2026',
               '07.02.2026', '09.03.2026', '01.10.2026', 'an sofort',
               '1.02.2026 (abhängig vom baufortschritt)', '10.04.2026',
               '13.01.2026', '25.01.2026', '07.03.2026', '17.01.2026',
               '24.01.2026', '06.02.2026', '31.05.2026', '1.12.2025',
               '28.01.2026', '27.01.2026', '17.02.2026', '15.04.2026',
               '18.02.2026', '31.03.2026', '21.02.2026', '05.03.2026',
               'sofort möglich', '22.02.2026', '15.06.2026', '06.04.2026',
               '14.03.2026', '08.03.2026', '03.02.2026', 'verfügbar',
               '24.02.2026', '11.02.2026', '25.02.2026', '16.03.2026',
               '04.02.2026', '26.03.2026', 'erstbezug', '10.03.2026',
               '30.10.2026', '27.02.2026'], dtype=object)
```

There are different dates and words / groups of words that mostly say that the date can be discussed or is immediately available and also the missing values (nan).

We could transform this feature into a number of days using the 'scraped\_at' feature (metadata from the scraper), but that would only work only if every post was scraped on the same day that it was posted, which was not possible.

The least we can do is transform the feature into a categorical feature using these 3 values:

- 'immediately' - convert the text values because they all say so
- 'not immediately' - dates, values that contain numbers like 'herbst 2025' or text that says 'depends' and similar.
- 'unknown' - nan values

```
In [36]: # Let's create a function to format these values based on what was suggested
```

```
def format_availability(value):
    # Missing values
    if pd.isna(value):
        return "unknown"

    # If already formatted
    if value in {"immediately", "not_immediately", "unknown"}:
        return value

    # Only strings
    if not isinstance(value, str):
        return "unknown"
```

```

v = value.strip().lower()

# "now / free / immediately" type signals
if re.search(r"\b(sofort|verfügbar|bezugsfrei|immediate)\b", v):
    return "immediately"

# not IMMEDIATE
# Any numeric reference → date or time
if re.search(r"\d", v):
    return "not_immediately"

# Temporal words (general, not specific dates)
if re.search(
    r"\b(ab|nach|bis|seit|herbst|winter|sommer|frühjahr|spring|summer|fall|autu
    v,
):
    return "not_immediately"

# If none of the cases then it's unknown
return "unknown"

```

```

In [37]: df["free_from"] = df["free_from"].apply(format_availability)
df["free_from"].value_counts()

```

```

Out[37]: free_from
unknown          8565
immediately      8251
not_immediately  4442
Name: count, dtype: int64

```

If you observe carefully, this feature had 8530 missing values and after formatting, 8565 were mapped as unknown, meaning there are almost no problematic values.

```

In [38]: df.isnull().sum()[df.isnull().sum() > 0]

```

```

Out[38]: floor          3122
location          326
flooring_type     13698
energy_source     12139
heating_type       9063
property_condition 12411
year_built        10233
dtype: int64

```

```

In [39]: df["year_built"].unique()

```

```
Out[39]: array(['2025', '1902', '2024', '2023', '2021', '2018', '2022', '1918',
                '2004', '1970', '2020', '2017', '2015', '1899', '1888', nan,
                '1954', '1900', '1986', '1911', '1995', '1908', '1905', '1980',
                '2026', '1910', '1906', '1992', '1994', '1912', '1890', '1938',
                '1950', '2019', '2010', '1903', '1935', '2016', '1998', '1955',
                '1960', '1996', '1977', '1978', '1965', '1920', '2000', '2007',
                '1976', '1930', '1989', '1982', '1988', '1962', '1997', '1894',
                '1880', '1993', '1979', '1961', '1974', '1904', '1967', '1963',
                '2003', '1984', '2011', '1946', '1969', '1966', '1957', '1892',
                '1959', '2005', '1907', '2006', '1878', '1971', '1956', '2008',
                '1864', '1973', '1896', '2001', '1765', '2009', '1913', '2014',
                '1968', '1915', '1929', '1958', '2012', '1925', '1934', '2013',
                '1936', '1916', '1928', '1983', '1990', '1895', '1919', '1999',
                '1953', 'neuwertig', '1901', '1937', '1870', '1926', '1985',
                '1927', '1012', '1887', '1839', '1949', '1972', '1850', '1877',
                '1975', '1907/2025', '1939', '1932', '1952', '1923', '2002',
                '1897', '1886', '1951', '1964', '1862', '2004/2018', '1893',
                '1933', '1991', '1914', '1875', '1931', '1874', '1909', '1898'],
                dtype=object)
```

```
In [40]: # we take the year built instead of year renovated in values like 'year_built/year_
# remove anomalies that could be cause by types like '1012' by replacing years earl
df["year_built"] = (
    df["year_built"]
    .astype(str)
    .str.extract(r"(\d{4})")[0]
    .astype("Int64")
)

# Remove implausible years
df.loc[df["year_built"] < 1700, "year_built"] = pd.NA
```

```
In [41]: df["year_built"].describe()
```

```
Out[41]: count      11014.0
mean       1997.833031
std         43.552269
min         1765.0
25%         1995.0
50%         2021.0
75%         2024.0
max         2026.0
Name: year_built, dtype: Float64
```

Now the data looks good and we keep nan values.

```
In [42]: df["flooring_type"].unique()
```

```

Out[42]: array(['fliesen, parkett', 'parkett', nan, 'holzdielen', 'teppich',
               'fliesen, kunststoff', 'fliesen, holzdielen, parkett', 'laminat',
               'fliesen', 'fliesen, parkett, granit', 'parkett, laminat',
               'fliesen, holzdielen', 'fliesen, laminat', 'kunststoff',
               'fliesen, laminat, terrakotta', 'holzdielen, kunststoff',
               'holzdielen, laminat, stein', 'holzdielen, parkett',
               'holzdielen, parkett, stein', 'fliesen, laminat, kunststoff',
               'laminat, kunststoff', 'fliesen, holzdielen, laminat',
               'parkett, kunststoff', 'holzdielen, laminat',
               'holzdielen, parkett, laminat', 'parkett, terrakotta',
               'kunststoff, stein', 'fliesen, parkett, laminat', 'linoleum',
               'marmor, holzdielen', 'holzdielen, parkett, terrakotta', 'stein',
               'holzdielen, stein', 'fliesen, parkett, kunststoff',
               'kunststoff, linoleum', 'fliesen, holzdielen, kunststoff',
               'fliesen, laminat, stein', 'marmor, laminat', 'parkett, stein',
               'fliesen, holzdielen, laminat, kunststoff',
               'holzdielen, terrakotta',
               'fliesen, holzdielen, parkett, kunststoff',
               'fliesen, parkett, stein', 'laminat, kunststoff, linoleum',
               'fliesen, holzdielen, kunststoff, linoleum', 'laminat, stein',
               'fliesen, teppich', 'fliesen, holzdielen, parkett, stein',
               'fliesen, laminat, stein, teppich', 'holzdielen, teppich',
               'fliesen, marmor, parkett', 'fliesen, linoleum, teppich',
               'fliesen, estrich, granit', 'marmor, parkett',
               'laminat, kunststoff, teppich', 'parkett, laminat, stein',
               'estrich', 'fliesen, parkett, teppich', 'parkett, doppelboden',
               'fliesen, holzdielen, parkett, laminat, kunststoff',
               'parkett, laminat, kunststoff',
               'fliesen, parkett, laminat, kunststoff', 'fliesen, linoleum',
               'fliesen, laminat, kunststoff, stein', 'fliesen, laminat, teppich',
               'laminat, kunststoff, stein', 'fliesen, holzdielen, terrakotta',
               'fliesen, parkett, laminat, teppich',
               'fliesen, parkett, kunststoff, teppich',
               'marmor, holzdielen, stein'], dtype=object)

```

```

In [43]: df["energy_source"].unique()

```

```
Out[43]: array(['Fernwärme', nan, 'Luft-/Wasser-Wärmepumpe', 'Gas', 'Öl',
               'Blockheizkraftwerk', 'Gas, Solar', 'Fernwärme, Elektro, Solar',
               'Elektro', 'Fernwärme, Luft-/Wasser-Wärmepumpe', 'Gas,',
               'Fernwärme, Gas, Blockheizkraftwerk', 'Solar', 'Erdwärme',
               'Elektro, Gas', 'Gas, Öl', 'Fernwärme, Öl',
               'Erdwärme, Luft-/Wasser-Wärmepumpe', 'Fernwärme, Erdwärme',
               'Elektro, Luft-/Wasser-Wärmepumpe',
               'Gas, Solar, Luft-/Wasser-Wärmepumpe', 'Fernwärme, Gas',
               'Gas, Blockheizkraftwerk', 'Solar, Blockheizkraftwerk',
               'Fernwärme, Elektro', 'Elektro, Solar', 'Gas, Flüssiggas',
               'Gas, Luft-/Wasser-Wärmepumpe', 'Fernwärme, Blockheizkraftwerk',
               'Pellets', 'Solar, Luft-/Wasser-Wärmepumpe',
               'Elektro, Solar, Luft-/Wasser-Wärmepumpe',
               'Gas, Flüssiggas, Solar', 'Gas, fossile Brennstoffe',
               'Elektro, Öl', 'Erdwärme, Solar, Luft-/Wasser-Wärmepumpe',
               'Kohle, Gas, Öl', 'Fernwärme, Elektro, Gas',
               'Elektro, Gas, Blockheizkraftwerk', 'fossile Brennstoffe',
               'Fernwärme, Elektro, Blockheizkraftwerk', 'Flüssiggas'],
              dtype=object)
```

```
In [44]: df["heating_type"].unique()
```

```
Out[44]: array(['Fußbodenheizung', 'Etagenheizung', 'Zentralheizung', nan,
               'Zentralheizung: Fußbodenheizung', 'individuell',
               'Zentralheizung: Heizkörper', 'Etagenheizung: Fußbodenheizung',
               'Ofen', 'individuell: Fußbodenheizung', 'individuell: Heizkörper'],
              dtype=object)
```

```
In [45]: df["property_condition"].unique()
```

```
Out[45]: array(['Erstbezug', nan, 'Altbau (bis 1945), neuwertig', 'neuwertig',
               'Neubau, Erstbezug', 'Neubau, neuwertig', 'renoviert / saniert',
               'Gepflegt', 'Altbau (bis 1945), renoviert / saniert', 'Neubau',
               'Modernisiert', 'Neubau, renoviert / saniert', 'Neubau, Gepflegt',
               'Altbau (bis 1945), Gepflegt', 'Altbau (bis 1945)', 'Saniert',
               'Massivhaus, neuwertig', 'Massivhaus',
               'Massivhaus, renoviert / saniert',
               'renovierungsbedürftig / sanierungsbedürftig',
               'Neubau, Massivhaus', 'Massivhaus, Gepflegt', 'teilsaniert',
               'Erstbezug nach Renovierung', 'Massivhaus, Erstbezug',
               'Erstbezug, renoviert / saniert', 'neuwertig, renoviert / saniert',
               'Gepflegt, renoviert / saniert', ', renoviert / saniert',
               'Neubau, Erstbezug, renoviert / saniert',
               'Neubau, neuwertig, renoviert / saniert',
               'Altbau (bis 1945), neuwertig, renoviert / saniert',
               'Saniert, renoviert / saniert', 'Fertighaus, renoviert / saniert',
               'Erstbezug nach Renovierung, renoviert / saniert',
               'Altbau (bis 1945), Saniert, renoviert / saniert',
               'Neubau, Massivhaus, Gepflegt'], dtype=object)
```

For the remaining 4 features: 'flooring\_type', 'energy\_source', 'heating\_type' and 'property\_condition', since they are all categorical features, we just replace missing values with 'Unknown' and then take care of them individually when we go through proper data formatting.



```
In [46]: df["flooring_type"] = df["flooring_type"].fillna("Unknown")
df["energy_source"] = df["energy_source"].fillna("Unknown")
df["heating_type"] = df["heating_type"].fillna("Unknown")
df["property_condition"] = df["property_condition"].fillna("Unknown")

df.isnull().sum()[df.isnull().sum() > 0]
```

```
Out[46]: floor          3122
location          326
year_built       10244
dtype: int64
```

```
In [47]: # We drop the samples with location missing and check how many samples we have.
df = df.dropna(subset=["location"])
df.shape
```

```
Out[47]: (20932, 24)
```

We just dropped (22687, 27) ==> (20932, 24) 1755 values and 3 columns

```
In [48]: 1755/22687
```

```
Out[48]: 0.07735707673998325
```

Losing only 7% of the data set is a very good achievement. Usually, I drop 10-20% of the data set, so we did really well in this one.

## Part 2: Handling duplications

Theoretically, we are not supposed to have duplicated records in this data set, because the scraper is implemented in such a way that it reads all values from stored URLs, stores them in a list and does not scrape them again even if they appear, but rather skips the link.

We still have to check for duplications because we cannot fully trust the scraper and also there might be listings that are reposted under a different listing, therefore having a different URL but the same data.

For these reasons, first we will check for duplicated URLs / records and then for records that might be duplicates.

```
In [49]: df.duplicated().any()
```

```
Out[49]: np.False_
```

No identical rows. That was expected since we have a unique feature: the 'scraped at'. Even if they have identical URLs, that's why we check the duplicated records excluding these 2 features.

```
In [50]: # exclude the URL and feature that serves as primary key
df.drop(columns=["listing_url", "scraped_at"]).duplicated().any()
```

Out[50]: np.True\_

```
In [51]: df[df.drop(columns=["listing_url", "scraped_at"]).duplicated()].head(5)
```

Out[51]:

|     | listing_url                                       | cold_rent | area              | rooms    | floor | fr      |
|-----|---------------------------------------------------|-----------|-------------------|----------|-------|---------|
| 119 | https://www.immowelt.de/expose/288c5511-4b4f-4... | 360 €     | 93 m <sup>2</sup> | 1 Zimmer | 0.0   | not_imn |
| 120 | https://www.immowelt.de/expose/6806cf09-9057-4... | 360 €     | 93 m <sup>2</sup> | 1 Zimmer | 0.0   | not_imn |
| 548 | https://www.immowelt.de/expose/25baabdc-7848-4... | 232 €     | 34 m <sup>2</sup> | 1 Zimmer | 0.0   | t       |
| 860 | https://www.immowelt.de/expose/aa61a4ba-c557-4... | 415 €     | 48 m <sup>2</sup> | 2 Zimmer | 0.0   | t       |
| 900 | https://www.immowelt.de/expose/e32e3e45-f2e6-4... | 500 €     | 45 m <sup>2</sup> | 2 Zimmer | 0.0   | t       |

5 rows × 24 columns



```
In [52]: # in case data has different formats
df["scraped_at"] = pd.to_datetime(df["scraped_at"], format="mixed", dayfirst=True)

# sort them in order to keep the newest versions
df = df.sort_values("scraped_at")

# drop duplicates with the earliest date and keep the latest version of the samples
df = df.drop_duplicates(
    subset=df.columns.difference(["listing_url", "scraped_at"]),
    keep="last"
)
```

```
In [53]: df.shape
```

Out[53]: (16021, 24)

20% of the dataset contained duplicated rows.

There might be a chance that there are almost identical duplicates, but with one value difference. In rare cases the scraper had scraped the same listing two times but failed to scrape all the values in one of the cases.

That is the reason why I want to check if they are duplicated rows without one or two of the other non-unique features.

```

In [54]: from itertools import combinations

# We totally exclude these columns
unique_cols = ["listing_url", "scraped_at"]

# We exclude the features that are more unique than the others like location, price
# There is no point checking if samples are unique without them
# We exclude them only from iteration, not from sample-unique-check
static_cols = ["location", "cold_rent", "area", "floor"]

# All columns we want to check for duplicates
compare_cols = [c for c in df.columns if c not in unique_cols]

# Columns we'll iterate through for exclusion (excluding the static ones)
iter_cols = [c for c in compare_cols if c not in static_cols]

# Number of columns to exclude at a time
X = 1

SENTINEL = "__MISSING__"
problematic_rows = set()
found_combinations = []

# Try all combinations of X columns to exclude (only from iter_cols)
for excluded_cols in combinations(iter_cols, X):
    # Columns to actually compare (all compare_cols except the excluded ones)
    cols_to_check = [c for c in compare_cols if c not in excluded_cols]

    # Find duplicates based on these columns
    temp_df = df[cols_to_check].astype("string").fillna(SENTINEL)
    dup_mask = temp_df.duplicated(keep=False)

    if dup_mask.any():
        dup_indices = df.index[dup_mask].tolist()
        problematic_rows.update(dup_indices)

        print(f"Excluded: {excluded_cols}")
        print(f" Found {dup_mask.sum()} duplicate rows")
        found_combinations.append({
            'excluded_cols': excluded_cols,
            'num_duplicates': dup_mask.sum(),
            'duplicate_indices': dup_indices
        })
        print()

print(f"\nTotal unique problematic rows: {len(problematic_rows)}")

```

Excluded: ('rooms',)  
Found 4 duplicate rows

Excluded: ('free\_from',)  
Found 86 duplicate rows

Excluded: ('has\_balkon',)  
Found 18 duplicate rows

Excluded: ('has\_terrasse',)  
Found 34 duplicate rows

Excluded: ('has\_garten',)  
Found 10 duplicate rows

Excluded: ('elevator',)  
Found 26 duplicate rows

Excluded: ('parking',)  
Found 8 duplicate rows

Excluded: ('has\_basement',)  
Found 12 duplicate rows

Excluded: ('is\_barrier-free',)  
Found 4 duplicate rows

Excluded: ('has\_built-in\_kitchen',)  
Found 4 duplicate rows

Excluded: ('has\_bathtub',)  
Found 4 duplicate rows

Excluded: ('has\_shower',)  
Found 6 duplicate rows

Excluded: ('flooring\_type',)  
Found 14 duplicate rows

Excluded: ('energy\_source',)  
Found 12 duplicate rows

Excluded: ('heating\_type',)  
Found 100 duplicate rows

Excluded: ('property\_condition',)  
Found 94 duplicate rows

Excluded: ('year\_built',)  
Found 10 duplicate rows

Total unique problematic rows: 440

```
In [55]: # Now we have to inspect all of these potential duplicates
df[df.drop(columns=["listing_url", "scraped_at", "rooms"]).duplicated(keep=False)]
```

Out[55]:

|       | listing_url                                       | cold_rent | area              | rooms    | floor |
|-------|---------------------------------------------------|-----------|-------------------|----------|-------|
| 209   | https://www.immowelt.de/expose/b056646c-dea4-4... | 1000 €    | 70 m²             | 2 Zimmer | 0.0   |
| 410   | https://www.immowelt.de/expose/6c28dbf7-34ed-4... | 1000 €    | 70 m²             | 3 Zimmer | 0.0   |
| 20311 | https://www.immowelt.de/expose/e1cf3cb9-28e9-4... | 2200 €    | 916 m² Grundstück | 4 Zimmer | NaN   |
| 21451 | https://www.immowelt.de/expose/cc8f124c-586b-4... | 2200 €    | 916 m² Grundstück | 3 Zimmer | NaN   |

4 rows × 24 columns



This is the perfect case to show that there were indeed duplicates with little change. We have 2 listings made within 2 days, possibly they deleted the original listing and posted it again later. Could be due to the wrong number of rooms, therefore we will assume that the one with the most recent date is the true sample.

```
In [56]: df = df.drop(index=[209, 20311])
```

```
In [57]: df[df.drop(columns=["listing_url", "scraped_at", "free_from"]).duplicated(keep=False)]
```

Out[57]:

|      | listing_url                                       | cold_rent | area              | rooms    | flo |
|------|---------------------------------------------------|-----------|-------------------|----------|-----|
| 3223 | https://www.immowelt.de/expose/c74aeffc-e6f4-4... | 771 €     | 52,8 m²           | 2 Zimmer | 1   |
| 3241 | https://www.immowelt.de/expose/c5afedd9-ed30-4... | 2650 €    | 300 m² Grundstück | 4 Zimmer | 1   |
| 3283 | https://www.immowelt.de/expose/1a86d9ed-9730-4... | 966 €     | 19 m²             | 1 Zimmer | 1   |
| 3285 | https://www.immowelt.de/expose/d8e40997-88aa-4... | 1239 €    | 32 m²             | 1 Zimmer | 1   |
| 3979 | https://www.immowelt.de/expose/c74aeffc-e6f4-4... | 771 €     | 52,8 m²           | 2 Zimmer | 1   |
| 3995 | https://www.immowelt.de/expose/c5afedd9-ed30-4... | 2650 €    | 300 m² Grundstück | 4 Zimmer | 1   |
| 4288 | https://www.immowelt.de/expose/653de956-40d3-4... | 1239 €    | 32 m²             | 1 Zimmer | 1   |
| 4342 | https://www.immowelt.de/expose/1a86d9ed-9730-4... | 966 €     | 19 m²             | 1 Zimmer | 1   |
| 5842 | https://www.immowelt.de/expose/a6b0219f-740f-4... | 2916.95 € | 158,1 m²          | 5 Zimmer | 2   |
| 5870 | https://www.immowelt.de/expose/25swdlasqtaq?se... | 1250 €    | 70 m²             | 2 Zimmer | 2   |

10 rows × 24 columns



In [58]:

```
# we drop the old ones and keep the most recent listing

subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "free_fro

print(f"Before: {df.shape[0]} samples")

df = (
    df
    .sort_values("scraped_at", ascending=False)
    .drop_duplicates(subset=subset, keep="first")
)
```

```
print(f"After: {df.shape[0]} samples")
```

Before: 16019 samples

After: 15976 samples

```
In [59]: df[df.drop(columns=["listing_url", "scraped_at", "has_balkon"]).duplicated(keep=False
```

Out[59]:

|              | listing_url                                                                                                       | cold_rent | area                | rooms    | floor |    |
|--------------|-------------------------------------------------------------------------------------------------------------------|-----------|---------------------|----------|-------|----|
| <b>21491</b> | <a href="https://www.immowelt.de/expose/6b9f4760-2e38-4...">https://www.immowelt.de/expose/6b9f4760-2e38-4...</a> | 1900 €    | 125 m <sup>2</sup>  | 4 Zimmer | NaN   | ir |
| <b>19487</b> | <a href="https://www.immowelt.de/expose/910aef75-a3dc-4...">https://www.immowelt.de/expose/910aef75-a3dc-4...</a> | 1900 €    | 125 m <sup>2</sup>  | 4 Zimmer | NaN   | ir |
| <b>16967</b> | <a href="https://www.immowelt.de/expose/910aef75-a3dc-4...">https://www.immowelt.de/expose/910aef75-a3dc-4...</a> | 1900 €    | 125 m <sup>2</sup>  | 4 Zimmer | 5.0   | ir |
| <b>16920</b> | <a href="https://www.immowelt.de/expose/6b9f4760-2e38-4...">https://www.immowelt.de/expose/6b9f4760-2e38-4...</a> | 1900 €    | 125 m <sup>2</sup>  | 4 Zimmer | 5.0   | ir |
| <b>11913</b> | <a href="https://www.immowelt.de/expose/910aef75-a3dc-4...">https://www.immowelt.de/expose/910aef75-a3dc-4...</a> | 1900 €    | 125 m <sup>2</sup>  | 4 Zimmer | 3.0   | ir |
| <b>11868</b> | <a href="https://www.immowelt.de/expose/6b9f4760-2e38-4...">https://www.immowelt.de/expose/6b9f4760-2e38-4...</a> | 1900 €    | 125 m <sup>2</sup>  | 4 Zimmer | 3.0   | ir |
| <b>7890</b>  | <a href="https://www.immowelt.de/expose/910aef75-a3dc-4...">https://www.immowelt.de/expose/910aef75-a3dc-4...</a> | 1900 €    | 125 m <sup>2</sup>  | 4 Zimmer | 2.0   | ir |
| <b>7837</b>  | <a href="https://www.immowelt.de/expose/6b9f4760-2e38-4...">https://www.immowelt.de/expose/6b9f4760-2e38-4...</a> | 1900 €    | 125 m <sup>2</sup>  | 4 Zimmer | 2.0   | ir |
| <b>5776</b>  | <a href="https://www.immowelt.de/expose/6e395239-885f-4...">https://www.immowelt.de/expose/6e395239-885f-4...</a> | 680 €     | 27,8 m <sup>2</sup> | 1 Zimmer | 1.0   | ir |
| <b>5135</b>  | <a href="https://www.immowelt.de/expose/6b9f4760-2e38-4...">https://www.immowelt.de/expose/6b9f4760-2e38-4...</a> | 1900 €    | 125 m <sup>2</sup>  | 4 Zimmer | 1.0   | ir |
| <b>5071</b>  | <a href="https://www.immowelt.de/expose/910aef75-a3dc-4...">https://www.immowelt.de/expose/910aef75-a3dc-4...</a> | 1900 €    | 125 m <sup>2</sup>  | 4 Zimmer | 1.0   | ir |



|      | listing_url                                                                                                       | cold_rent | area    | rooms    | floor |        |
|------|-------------------------------------------------------------------------------------------------------------------|-----------|---------|----------|-------|--------|
| 4405 | <a href="https://www.immowelt.de/expose/c4e61499-bc1f-4...">https://www.immowelt.de/expose/c4e61499-bc1f-4...</a> | 765 €     | 32 m²   | 1 Zimmer | 1.0   | not_ir |
| 4396 | <a href="https://www.immowelt.de/expose/5df5bd39-e128-4...">https://www.immowelt.de/expose/5df5bd39-e128-4...</a> | 966.67 €  | 68,5 m² | 2 Zimmer | 1.0   | ir     |
| 4095 | <a href="https://www.immowelt.de/expose/f3800ef4-be39-4...">https://www.immowelt.de/expose/f3800ef4-be39-4...</a> | 431.72 €  | 42,1 m² | 1 Zimmer | 1.0   | ir     |
| 4039 | <a href="https://www.immowelt.de/expose/aee1f5f1-5af4-4...">https://www.immowelt.de/expose/aee1f5f1-5af4-4...</a> | 680 €     | 27,8 m² | 1 Zimmer | 1.0   | ir     |
| 3451 | <a href="https://www.immowelt.de/expose/9da55270-dfcb-4...">https://www.immowelt.de/expose/9da55270-dfcb-4...</a> | 765 €     | 32 m²   | 1 Zimmer | 1.0   | not_ir |
| 3280 | <a href="https://www.immowelt.de/expose/5df5bd39-e128-4...">https://www.immowelt.de/expose/5df5bd39-e128-4...</a> | 966.67 €  | 68,5 m² | 2 Zimmer | 1.0   | ir     |
| 3237 | <a href="https://www.immowelt.de/expose/f3800ef4-be39-4...">https://www.immowelt.de/expose/f3800ef4-be39-4...</a> | 431.72 €  | 42,1 m² | 1 Zimmer | 1.0   | ir     |

18 rows × 24 columns

```
In [60]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "has_balk
print(f"Before: {df.shape[0]} samples")

df = (
    df
    .sort_values("scraped_at", ascending=False)
    .drop_duplicates(subset=subset, keep="first")
)

print(f"After: {df.shape[0]} samples")
```

Before: 15976 samples

After: 15967 samples

```
In [61]: df[df.drop(columns=["listing_url", "scraped_at", "has_terrasse"]).duplicated(keep=False)]
```

Out[61]:

|      | listing_url                                                                                                       | cold_rent | area                 | rooms    | floor |        |
|------|-------------------------------------------------------------------------------------------------------------------|-----------|----------------------|----------|-------|--------|
| 5714 | <a href="https://www.immowelt.de/expose/4da5fdaf-a792-4...">https://www.immowelt.de/expose/4da5fdaf-a792-4...</a> | 2299 €    | 100,5 m <sup>2</sup> | 3 Zimmer | 1.0   | ir     |
| 5710 | <a href="https://www.immowelt.de/expose/afa02248-6b1f-4...">https://www.immowelt.de/expose/afa02248-6b1f-4...</a> | 2480 €    | 84,5 m <sup>2</sup>  | 4 Zimmer | 1.0   | ir     |
| 5701 | <a href="https://www.immowelt.de/expose/82da02a1-d358-4...">https://www.immowelt.de/expose/82da02a1-d358-4...</a> | 1549 €    | 60,1 m <sup>2</sup>  | 2 Zimmer | 1.0   | not_ir |
| 5700 | <a href="https://www.immowelt.de/expose/1d3ece94-51bf-4...">https://www.immowelt.de/expose/1d3ece94-51bf-4...</a> | 1469 €    | 55,5 m <sup>2</sup>  | 2 Zimmer | 1.0   | not_ir |
| 5699 | <a href="https://www.immowelt.de/expose/31e16e72-7fab-4...">https://www.immowelt.de/expose/31e16e72-7fab-4...</a> | 2050 €    | 87,5 m <sup>2</sup>  | 3 Zimmer | 1.0   | not_ir |
| 5663 | <a href="https://www.immowelt.de/expose/abecabb0-2fab-4...">https://www.immowelt.de/expose/abecabb0-2fab-4...</a> | 2349 €    | 121,4 m <sup>2</sup> | 4 Zimmer | 1.0   | ir     |
| 5653 | <a href="https://www.immowelt.de/expose/8d9c9ef0-3a2f-4...">https://www.immowelt.de/expose/8d9c9ef0-3a2f-4...</a> | 3239 €    | 178,3 m <sup>2</sup> | 4 Zimmer | 1.0   | not_ir |
| 5652 | <a href="https://www.immowelt.de/expose/2968d626-7614-4...">https://www.immowelt.de/expose/2968d626-7614-4...</a> | 1189 €    | 43,4 m <sup>2</sup>  | 2 Zimmer | 1.0   | not_ir |
| 5638 | <a href="https://www.immowelt.de/expose/c7e7a447-8aab-4...">https://www.immowelt.de/expose/c7e7a447-8aab-4...</a> | 995 €     | 36,2 m <sup>2</sup>  | 1 Zimmer | 1.0   | ir     |
| 5633 | <a href="https://www.immowelt.de/expose/7a0b51f4-5e93-4...">https://www.immowelt.de/expose/7a0b51f4-5e93-4...</a> | 1675 €    | 69,2 m <sup>2</sup>  | 3 Zimmer | 1.0   | ir     |
| 5611 | <a href="https://www.immowelt.de/expose/c7e7a447-8aab-4...">https://www.immowelt.de/expose/c7e7a447-8aab-4...</a> | 995 €     | 36,2 m <sup>2</sup>  | 1 Zimmer | 1.0   | ir     |
| 5606 | <a href="https://www.immowelt.de/expose/7a0b51f4-5e93-4...">https://www.immowelt.de/expose/7a0b51f4-5e93-4...</a> | 1675 €    | 69,2 m <sup>2</sup>  | 3 Zimmer | 1.0   | ir     |

|      | listing_url                                                                                                       | cold_rent | area        | rooms       | floor |        |
|------|-------------------------------------------------------------------------------------------------------------------|-----------|-------------|-------------|-------|--------|
| 5597 | <a href="https://www.immowelt.de/expose/abecabb0-2fab-4...">https://www.immowelt.de/expose/abecabb0-2fab-4...</a> | 2349 €    | 121,4<br>m² | 4<br>Zimmer | 1.0   | ir     |
| 5595 | <a href="https://www.immowelt.de/expose/971b5f79-ce03-4...">https://www.immowelt.de/expose/971b5f79-ce03-4...</a> | 2299 €    | 100,5<br>m² | 3<br>Zimmer | 1.0   | ir     |
| 5581 | <a href="https://www.immowelt.de/expose/8d9c9ef0-3a2f-4...">https://www.immowelt.de/expose/8d9c9ef0-3a2f-4...</a> | 3239 €    | 178,3<br>m² | 4<br>Zimmer | 1.0   | not_ir |
| 5580 | <a href="https://www.immowelt.de/expose/2968d626-7614-4...">https://www.immowelt.de/expose/2968d626-7614-4...</a> | 1189 €    | 43,4<br>m²  | 2<br>Zimmer | 1.0   | not_ir |
| 5500 | <a href="https://www.immowelt.de/expose/82da02a1-d358-4...">https://www.immowelt.de/expose/82da02a1-d358-4...</a> | 1549 €    | 60,1<br>m²  | 2<br>Zimmer | 1.0   | not_ir |
| 5499 | <a href="https://www.immowelt.de/expose/1d3ece94-51bf-4...">https://www.immowelt.de/expose/1d3ece94-51bf-4...</a> | 1469 €    | 55,5<br>m²  | 2<br>Zimmer | 1.0   | not_ir |
| 5498 | <a href="https://www.immowelt.de/expose/31e16e72-7fab-4...">https://www.immowelt.de/expose/31e16e72-7fab-4...</a> | 2050 €    | 87,5<br>m²  | 3<br>Zimmer | 1.0   | not_ir |
| 5457 | <a href="https://www.immowelt.de/expose/afa02248-6b1f-4...">https://www.immowelt.de/expose/afa02248-6b1f-4...</a> | 2480 €    | 84,5<br>m²  | 4<br>Zimmer | 1.0   | ir     |
| 5162 | <a href="https://www.immowelt.de/expose/270862dc-a9a1-4...">https://www.immowelt.de/expose/270862dc-a9a1-4...</a> | 1169 €    | 51,2<br>m²  | 2<br>Zimmer | 1.0   | not_ir |
| 5097 | <a href="https://www.immowelt.de/expose/7bbd68d8-b447-4...">https://www.immowelt.de/expose/7bbd68d8-b447-4...</a> | 1150 €    | 74<br>m²    | 2<br>Zimmer | 1.0   | ir     |
| 5070 | <a href="https://www.immowelt.de/expose/aceecbf2-4776-4...">https://www.immowelt.de/expose/aceecbf2-4776-4...</a> | 1150 €    | 74<br>m²    | 2<br>Zimmer | 1.0   | ir     |

|      | listing_url                                                                                                       | cold_rent | area       | rooms       | floor |        |
|------|-------------------------------------------------------------------------------------------------------------------|-----------|------------|-------------|-------|--------|
| 4994 | <a href="https://www.immowelt.de/expose/428c0787-7dd8-4...">https://www.immowelt.de/expose/428c0787-7dd8-4...</a> | 1169 €    | 51,2<br>m² | 2<br>Zimmer | 1.0   | not_ir |
| 4898 | <a href="https://www.immowelt.de/expose/52561aa8-0ffa-4...">https://www.immowelt.de/expose/52561aa8-0ffa-4...</a> | 1025 €    | 36,7<br>m² | 1<br>Zimmer | 1.0   | not_ir |
| 4807 | <a href="https://www.immowelt.de/expose/b089bf00-2f80-4...">https://www.immowelt.de/expose/b089bf00-2f80-4...</a> | 897 €     | 48,5<br>m² | 2<br>Zimmer | 1.0   | not_ir |
| 4332 | <a href="https://www.immowelt.de/expose/6c0cd9c1-7a37-4...">https://www.immowelt.de/expose/6c0cd9c1-7a37-4...</a> | 988.24 €  | 62,1<br>m² | 2<br>Zimmer | 1.0   | ir     |
| 3999 | <a href="https://www.immowelt.de/expose/f5530ea4-d5a0-4...">https://www.immowelt.de/expose/f5530ea4-d5a0-4...</a> | 1439.4 €  | 96<br>m²   | 2<br>Zimmer | 1.0   | ir     |
| 3998 | <a href="https://www.immowelt.de/expose/8bb0f98f-88d4-4...">https://www.immowelt.de/expose/8bb0f98f-88d4-4...</a> | 1642 €    | 78<br>m²   | 2<br>Zimmer | 1.0   | ir     |
| 3624 | <a href="https://www.immowelt.de/expose/02aff49f-e60b-4...">https://www.immowelt.de/expose/02aff49f-e60b-4...</a> | 1025 €    | 36,7<br>m² | 1<br>Zimmer | 1.0   | not_ir |
| 3570 | <a href="https://www.immowelt.de/expose/b364e497-a8a7-4...">https://www.immowelt.de/expose/b364e497-a8a7-4...</a> | 897 €     | 48,5<br>m² | 2<br>Zimmer | 1.0   | not_ir |
| 3264 | <a href="https://www.immowelt.de/expose/6c0cd9c1-7a37-4...">https://www.immowelt.de/expose/6c0cd9c1-7a37-4...</a> | 988.24 €  | 62,1<br>m² | 2<br>Zimmer | 1.0   | ir     |
| 3157 | <a href="https://www.immowelt.de/expose/f5530ea4-d5a0-4...">https://www.immowelt.de/expose/f5530ea4-d5a0-4...</a> | 1439.4 €  | 96<br>m²   | 2<br>Zimmer | 1.0   | ir     |
| 2862 | <a href="https://www.immowelt.de/expose/8bb0f98f-88d4-4...">https://www.immowelt.de/expose/8bb0f98f-88d4-4...</a> | 1642 €    | 78<br>m²   | 2<br>Zimmer | 1.0   | ir     |

34 rows × 24 columns

```
In [62]: # We keep the ones with true 'has_tarrase'
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "has_terr

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_terrace", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

After: 15950 samples

```
In [63]: df[df.drop(columns=["listing_url", "scraped_at", "has_garten"]).duplicated(keep=False
```

Out[63]:

|             | listing_url                                                                                                       | cold_rent | area                | rooms    | floor |        |
|-------------|-------------------------------------------------------------------------------------------------------------------|-----------|---------------------|----------|-------|--------|
| <b>4001</b> | <a href="https://www.immowelt.de/expose/3f918689-673a-4...">https://www.immowelt.de/expose/3f918689-673a-4...</a> | 650 €     | 45 m <sup>2</sup>   | 2 Zimmer | 1.0   |        |
| <b>4110</b> | <a href="https://www.immowelt.de/expose/49d10714-984a-4...">https://www.immowelt.de/expose/49d10714-984a-4...</a> | 650 €     | 60 m <sup>2</sup>   | 2 Zimmer | 1.0   |        |
| <b>2923</b> | <a href="https://www.immowelt.de/expose/3f918689-673a-4...">https://www.immowelt.de/expose/3f918689-673a-4...</a> | 650 €     | 45 m <sup>2</sup>   | 2 Zimmer | 1.0   |        |
| <b>2977</b> | <a href="https://www.immowelt.de/expose/49d10714-984a-4...">https://www.immowelt.de/expose/49d10714-984a-4...</a> | 650 €     | 60 m <sup>2</sup>   | 2 Zimmer | 1.0   |        |
| <b>3575</b> | <a href="https://www.immowelt.de/expose/b2efac81-2bca-4...">https://www.immowelt.de/expose/b2efac81-2bca-4...</a> | 431 €     | 16 m <sup>2</sup>   | 1 Zimmer | 1.0   | not_im |
| <b>3572</b> | <a href="https://www.immowelt.de/expose/535a8a2f-a240-4...">https://www.immowelt.de/expose/535a8a2f-a240-4...</a> | 1056 €    | 53,7 m <sup>2</sup> | 2 Zimmer | 1.0   | not_im |
| <b>3512</b> | <a href="https://www.immowelt.de/expose/48940456-e12b-4...">https://www.immowelt.de/expose/48940456-e12b-4...</a> | 1030 €    | 42 m <sup>2</sup>   | 1 Zimmer | 1.0   | not_im |
| <b>4735</b> | <a href="https://www.immowelt.de/expose/9696b5f0-551b-4...">https://www.immowelt.de/expose/9696b5f0-551b-4...</a> | 1030 €    | 42 m <sup>2</sup>   | 1 Zimmer | 1.0   | not_im |
| <b>4831</b> | <a href="https://www.immowelt.de/expose/696e7f4b-6034-4...">https://www.immowelt.de/expose/696e7f4b-6034-4...</a> | 431 €     | 16 m <sup>2</sup>   | 1 Zimmer | 1.0   | not_im |
| <b>4808</b> | <a href="https://www.immowelt.de/expose/1cb45d7f-486c-4...">https://www.immowelt.de/expose/1cb45d7f-486c-4...</a> | 1056 €    | 53,7 m <sup>2</sup> | 2 Zimmer | 1.0   | not_im |

10 rows × 24 columns

```
In [64]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "has_gart"]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_garten", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

After: 15945 samples

For boolean values we won't inspect the samples anymore, we will just immediately drop them.

```
In [65]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "elevator"]

df = (
    df
    # True first, then False, then NaN
    .sort_values("elevator", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

After: 15932 samples

```
In [66]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "parking"]

df = (
    df
    # True first, then False, then NaN
    .sort_values("parking", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

After: 15928 samples

```
In [67]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "has_base"]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_basement", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

After: 15922 samples

```
In [68]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "is_barri"]
```

```

df = (
    df
    # True first, then False, then NaN
    .sort_values("is_barrier-free", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")

```

After: 15920 samples

```

In [69]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "has_built_in_kitchen"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_built-in_kitchen", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")

```

After: 15918 samples

```

In [70]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "has_bath_tub"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_bathtub", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")

```

After: 15916 samples

```

In [71]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "has_showers"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_shower", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")

```

After: 15913 samples

```

In [72]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "flooring_type"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("flooring_type", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")

```

After: 15908 samples



```
In [73]: df[df.drop(columns=["listing_url", "scraped_at", "energy_source"]).duplicated(keep=False)]
```

Out[73]:

|      | listing_url                                                                                                       | cold_rent | area                | rooms    | floor |        |
|------|-------------------------------------------------------------------------------------------------------------------|-----------|---------------------|----------|-------|--------|
| 6386 | <a href="https://www.immowelt.de/expose/05310f59-6821-4...">https://www.immowelt.de/expose/05310f59-6821-4...</a> | 665 €     | 26,5 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |
| 7629 | <a href="https://www.immowelt.de/expose/23007f52-c757-4...">https://www.immowelt.de/expose/23007f52-c757-4...</a> | 730 €     | 29,2 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |
| 6378 | <a href="https://www.immowelt.de/expose/64d88100-c1b6-4...">https://www.immowelt.de/expose/64d88100-c1b6-4...</a> | 700 €     | 27,4 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |
| 6379 | <a href="https://www.immowelt.de/expose/4435f29c-4172-4...">https://www.immowelt.de/expose/4435f29c-4172-4...</a> | 700 €     | 27,8 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |
| 7533 | <a href="https://www.immowelt.de/expose/4435f29c-4172-4...">https://www.immowelt.de/expose/4435f29c-4172-4...</a> | 700 €     | 27,8 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |
| 7609 | <a href="https://www.immowelt.de/expose/a6fc625a-beb4-4...">https://www.immowelt.de/expose/a6fc625a-beb4-4...</a> | 625 €     | 24,9 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |
| 7532 | <a href="https://www.immowelt.de/expose/64d88100-c1b6-4...">https://www.immowelt.de/expose/64d88100-c1b6-4...</a> | 700 €     | 27,4 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |
| 5833 | <a href="https://www.immowelt.de/expose/23007f52-c757-4...">https://www.immowelt.de/expose/23007f52-c757-4...</a> | 730 €     | 29,2 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |
| 7542 | <a href="https://www.immowelt.de/expose/05310f59-6821-4...">https://www.immowelt.de/expose/05310f59-6821-4...</a> | 665 €     | 26,5 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |

|      | listing_url                                       | cold_rent | area                | rooms    | floor |        |
|------|---------------------------------------------------|-----------|---------------------|----------|-------|--------|
| 6211 | https://www.immowelt.de/expose/a6fc625a-beb4-4... | 625 €     | 24,9 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |
| 7610 | https://www.immowelt.de/expose/45f0109a-a961-4... | 740 €     | 29,5 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |
| 6227 | https://www.immowelt.de/expose/45f0109a-a961-4... | 740 €     | 29,5 m <sup>2</sup> | 1 Zimmer | 2.0   | not_im |

12 rows × 24 columns

```
In [74]: # We keep the samples with that have the 'energy_source' value and drop those with
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "energy_s

to_drop = df[
    df.duplicated(subset=subset, keep=False) &
    (df["energy_source"] == "Unknown")
].index

df = df.drop(index=to_drop, errors="ignore")
```

```
In [75]: print(f"After: {df.shape[0]} samples")
```

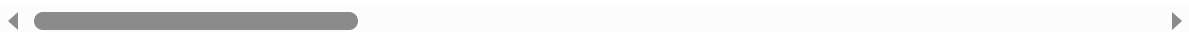
After: 15902 samples

```
In [76]: df[df.drop(columns=["listing_url", "scraped_at", "heating_type"]).duplicated(keep=False)]
```

Out[76]:

|              | listing_url                                                                                                       | cold_rent | area                | rooms    | floor |        |
|--------------|-------------------------------------------------------------------------------------------------------------------|-----------|---------------------|----------|-------|--------|
| <b>3409</b>  | <a href="https://www.immowelt.de/expose/43778d73-a328-4...">https://www.immowelt.de/expose/43778d73-a328-4...</a> | 1149 €    | 60,5 m <sup>2</sup> | 2 Zimmer | 1.0   | ir     |
| <b>4310</b>  | <a href="https://www.immowelt.de/expose/43778d73-a328-4...">https://www.immowelt.de/expose/43778d73-a328-4...</a> | 1149 €    | 60,5 m <sup>2</sup> | 2 Zimmer | 1.0   | ir     |
| <b>11328</b> | <a href="https://www.immowelt.de/expose/8404f011-be75-4...">https://www.immowelt.de/expose/8404f011-be75-4...</a> | 574.8 €   | 18,8 m <sup>2</sup> | 1 Zimmer | 3.0   | not_ir |
| <b>11327</b> | <a href="https://www.immowelt.de/expose/8753be53-a399-4...">https://www.immowelt.de/expose/8753be53-a399-4...</a> | 1013.24 € | 48 m <sup>2</sup>   | 2 Zimmer | 3.0   | not_ir |
| <b>12192</b> | <a href="https://www.immowelt.de/expose/dfbd4c17-bd71-4...">https://www.immowelt.de/expose/dfbd4c17-bd71-4...</a> | 532.6 €   | 18,1 m <sup>2</sup> | 1 Zimmer | 3.0   | not_ir |
| <b>11325</b> | <a href="https://www.immowelt.de/expose/1bd95b25-231b-4...">https://www.immowelt.de/expose/1bd95b25-231b-4...</a> | 460.6 €   | 23,6 m <sup>2</sup> | 1 Zimmer | 3.0   | not_ir |
| <b>12146</b> | <a href="https://www.immowelt.de/expose/73c79502-aa68-4...">https://www.immowelt.de/expose/73c79502-aa68-4...</a> | 530 €     | 11,8 m <sup>2</sup> | 1 Zimmer | 3.0   | not_ir |
| <b>12987</b> | <a href="https://www.immowelt.de/expose/6a211cbc-1803-4...">https://www.immowelt.de/expose/6a211cbc-1803-4...</a> | 525 €     | 11 m <sup>2</sup>   | 1 Zimmer | 3.0   | not_ir |
| <b>11320</b> | <a href="https://www.immowelt.de/expose/567f9a33-d4d7-4...">https://www.immowelt.de/expose/567f9a33-d4d7-4...</a> | 917 €     | 49 m <sup>2</sup>   | 2 Zimmer | 3.0   | not_ir |
| <b>5705</b>  | <a href="https://www.immowelt.de/expose/a821c056-d519-4...">https://www.immowelt.de/expose/a821c056-d519-4...</a> | 518 €     | 11,3 m <sup>2</sup> | 1 Zimmer | 1.0   | not_ir |

10 rows × 24 columns



```
In [77]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "heating_to_drop = df[
    df.duplicated(subset=subset, keep=False) &
    (df["heating_type"] == "Unknown")
].index
df = df.drop(index=to_drop, errors="ignore")
```

```
In [78]: df[df.drop(columns=["listing_url", "scraped_at", "property_condition"]).duplicated(
```

Out[78]:

|       | listing_url                                                                                                       | cold_rent | area    | rooms    | floor | free |
|-------|-------------------------------------------------------------------------------------------------------------------|-----------|---------|----------|-------|------|
| 5736  | <a href="https://www.immowelt.de/expose/b1f1fe89-c0b1-4...">https://www.immowelt.de/expose/b1f1fe89-c0b1-4...</a> | 1239 €    | 51 m²   | 2 Zimmer | 1.0   | imme |
| 5432  | <a href="https://www.immowelt.de/expose/b1f1fe89-c0b1-4...">https://www.immowelt.de/expose/b1f1fe89-c0b1-4...</a> | 1239 €    | 51 m²   | 2 Zimmer | 1.0   | imme |
| 17058 | <a href="https://www.immowelt.de/expose/9ee635c2-3f38-4...">https://www.immowelt.de/expose/9ee635c2-3f38-4...</a> | 1170 €    | 36 m²   | 1 Zimmer | 5.0   | imme |
| 17327 | <a href="https://www.immowelt.de/expose/53cdcf05-0953-4...">https://www.immowelt.de/expose/53cdcf05-0953-4...</a> | 1170 €    | 36 m²   | 1 Zimmer | 5.0   | imme |
| 5718  | <a href="https://www.immowelt.de/expose/3ffdf53b-a483-4...">https://www.immowelt.de/expose/3ffdf53b-a483-4...</a> | 1650 €    | 82,2 m² | 3 Zimmer | 1.0   | imme |
| 11763 | <a href="https://www.immowelt.de/expose/283e71a5-6467-4...">https://www.immowelt.de/expose/283e71a5-6467-4...</a> | 1700 €    | 70 m²   | 2 Zimmer | 3.0   | imme |
| 13004 | <a href="https://www.immowelt.de/expose/283e71a5-6467-4...">https://www.immowelt.de/expose/283e71a5-6467-4...</a> | 1700 €    | 70 m²   | 2 Zimmer | 3.0   | imme |
| 5470  | <a href="https://www.immowelt.de/expose/3ffdf53b-a483-4...">https://www.immowelt.de/expose/3ffdf53b-a483-4...</a> | 1650 €    | 82,2 m² | 3 Zimmer | 1.0   | imme |
| 4335  | <a href="https://www.immowelt.de/expose/f0d38787-0c2f-4...">https://www.immowelt.de/expose/f0d38787-0c2f-4...</a> | 400 €     | 36 m²   | 1 Zimmer | 1.0   | un   |
| 3095  | <a href="https://www.immowelt.de/expose/f0d38787-0c2f-4...">https://www.immowelt.de/expose/f0d38787-0c2f-4...</a> | 400 €     | 36 m²   | 1 Zimmer | 1.0   | un   |

10 rows × 24 columns



```
In [79]: subset = [c for c in df.columns if c not in ["listing_url", "scraped_at", "property"]
df["scraped_at"] = pd.to_datetime(df["scraped_at"], errors="coerce")

df = (
    df
    # prefer non-Unknown
```

```

        .assign(_known=lambda x: x["property_condition"].ne("Unknown"))
        # sort so best row comes first within each duplicate group:
        # 1) known property_condition (True first)
        # 2) most recent scraped_at (newest first)
        .sort_values(["_known", "scraped_at"], ascending=[False, False], na_position="last")
        # keep one per group
        .drop_duplicates(subset="property_condition", keep="first")
        .drop(columns=["_known"])
    )

    print(f"After: {df.shape[0]} samples")

```

After: 15808 samples

```

In [80]: df[df.drop(columns=["listing_url", "scraped_at", "year_built"]).duplicated(keep=False)]

```

Out[80]:

|      | listing_url                                       | cold_rent | area    | rooms    | floor |    |
|------|---------------------------------------------------|-----------|---------|----------|-------|----|
| 7989 | https://www.immowelt.de/expose/2554ujfn17u6?se... | 1090 €    | 77,6 m² | 2 Zimmer | 2.0   | no |
| 5919 | https://www.immowelt.de/expose/2554ujfn17u6?se... | 1090 €    | 77,6 m² | 2 Zimmer | 2.0   | no |
| 7617 | https://www.immowelt.de/expose/fe0d7373-7586-4... | 360 €     | 93 m²   | 1 Zimmer | 2.0   | no |
| 7539 | https://www.immowelt.de/expose/043525f5-c584-4... | 1287 €    | 48 m²   | 2 Zimmer | 2.0   | no |
| 7531 | https://www.immowelt.de/expose/b102b2ac-c8da-4... | 526.4 €   | 18,4 m² | 1 Zimmer | 2.0   | no |
| 6374 | https://www.immowelt.de/expose/11c74691-d14e-4... | 1200 €    | 49,3 m² | 2 Zimmer | 2.0   | no |
| 6039 | https://www.immowelt.de/expose/d395cd36-eae0-4... | 1200 €    | 49,3 m² | 2 Zimmer | 2.0   | no |
| 6029 | https://www.immowelt.de/expose/936dbfc8-a050-4... | 360 €     | 93 m²   | 1 Zimmer | 2.0   | no |
| 6012 | https://www.immowelt.de/expose/1a7ea234-41d5-4... | 1287 €    | 48 m²   | 2 Zimmer | 2.0   | no |
| 6011 | https://www.immowelt.de/expose/18f98470-b59d-4... | 526.4 €   | 18,4 m² | 1 Zimmer | 2.0   | no |

10 rows × 24 columns



In [81]: `df[df.duplicated(subset=[c for c in df.columns if c not in ["listing_url", "scraped`

Out[81]:

|      | listing_url                                       | cold_rent | area    | rooms    | floor |        |
|------|---------------------------------------------------|-----------|---------|----------|-------|--------|
| 6039 | https://www.immowelt.de/expose/d395cd36-eae0-4... | 1200 €    | 49,3 m² | 2 Zimmer | 2.0   | not_im |
| 6029 | https://www.immowelt.de/expose/936dbfc8-a050-4... | 360 €     | 93 m²   | 1 Zimmer | 2.0   | not_im |
| 6012 | https://www.immowelt.de/expose/1a7ea234-41d5-4... | 1287 €    | 48 m²   | 2 Zimmer | 2.0   | not_im |
| 6011 | https://www.immowelt.de/expose/18f98470-b59d-4... | 526.4 €   | 18,4 m² | 1 Zimmer | 2.0   | not_im |

4 rows × 24 columns



```
In [82]: # We drop them those 4 with NA, but we would be left with 1 duplicate since there w
df = df.drop(index=df[df.duplicated(subset=[c for c in df.columns if c not in ["lis
```

```
In [83]: df[df.drop(columns=["listing_url", "scraped_at", "year_built"]).duplicated(keep=False
```

Out[83]:

|      | listing_url                                       | cold_rent | area    | rooms    | floor |    |
|------|---------------------------------------------------|-----------|---------|----------|-------|----|
| 7989 | https://www.immowelt.de/expose/2554ujfn17u6?se... | 1090 €    | 77,6 m² | 2 Zimmer | 2.0   | no |
| 5919 | https://www.immowelt.de/expose/2554ujfn17u6?se... | 1090 €    | 77,6 m² | 2 Zimmer | 2.0   | no |

2 rows × 24 columns



```
In [84]: # We keep the one with the original year (Which also happens to be the newer Listin
df = df.drop(index=[5919])
```

```
In [85]: df.shape
```

Out[85]: (15803, 24)



During deduplication we dropped from 20932 to 15803  
22687 ==> 20932 ==> 15803

```
In [86]: (1-(15803/22687))*100
```

```
Out[86]: 30.34336844889144
```

We lost 30.34% of our dataset

```
In [87]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 15803 entries, 22653 to 3
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   listing_url            15803 non-null  object
1   cold_rent              15803 non-null  object
2   area                  15803 non-null  object
3   rooms                 15803 non-null  object
4   floor                 13832 non-null  float64
5   free_from             15803 non-null  object
6   location              15803 non-null  object
7   has_balkon            15803 non-null  boolean
8   has_terrasse          15803 non-null  boolean
9   has_garten            15803 non-null  boolean
10  elevator              15803 non-null  boolean
11  parking               15803 non-null  boolean
12  has_basement          15803 non-null  boolean
13  is_barrier-free       15803 non-null  boolean
14  has_built-in_kitchen  15803 non-null  boolean
15  has_bathtub           15803 non-null  boolean
16  has_shower            15803 non-null  boolean
17  flooring_type         15803 non-null  object
18  energy_source         15803 non-null  object
19  heating_type          15803 non-null  object
20  property_condition    15803 non-null  object
21  year_built            7839 non-null   Int64
22  schufa_check          15803 non-null  bool
23  scraped_at            15803 non-null  datetime64[ns]
dtypes: Int64(1), bool(1), boolean(10), datetime64[ns](1), float64(1), object(10)
memory usage: 2.0+ MB
```

Only the year built and the floor type have missing values

## Part 3: Handling inconsistent formats

### 1. Formatting 'cold rent' feature

```
In [88]: df["cold_rent"].sample(5)
```

```
Out[88]: 1407      380 €
          8074      479 €
          8066    1003.8 €
          8522   1440.03 €
          18268    1102 €
          Name: cold_rent, dtype: object
```

```
In [89]: # check if there are any irregular values (values other than 'number + currency')
pattern = r'^\s*(?:\d{1,3}(?:[.]\d{3})+|\d+)(?:[.]\d+)?\s*€?\s*$'
mask = df['cold_rent'].astype(str).str.match(pattern)
df_invalid = df[~mask]
df_invalid['cold_rent']
```

```
Out[89]: Series([], Name: cold_rent, dtype: object)
```

```
In [90]: # Let's also check if any of the values have comma in case we need to replace it with dot
mask = df['cold_rent'].astype(str).str.contains(',')
df_invalid = df[mask]
df_invalid['cold_rent'].sample(5)
```

```
Out[90]: 26      1,449 €
          20    1,586.85 €
          13      2,110 €
          15      1,095 €
          17      1,595 €
          Name: cold_rent, dtype: object
```

```
In [91]: # Let's format the column so it contains only the numerical values
s = (df['cold_rent'].astype(str)
      .str.strip()
      .str.replace(r'[€\s]', '', regex=True))

# Sampled values had US format, but it is recommended to ensure to treat both
us = s.str.contains(r'\d{3}(\D|$)') & s.str.contains(r'\.\d+$') # 1,234.56
eu = s.str.contains(r'\.\d{3}(\D|$)') & s.str.contains(r'\d+$') # 1.234,56

out = pd.Series(index=s.index, dtype='float64')

# US: remove thousands commas
out[us] = pd.to_numeric(s[us].str.replace(',', '', regex=False), errors='coerce')

# EU: remove thousands dots, swap decimal comma -> dot
out[eu] = pd.to_numeric(
    s[eu].str.replace('.', '', regex=False).str.replace(',', '.', regex=False),
    errors='coerce'
)

# Plain: no thousands separators, maybe dot-decimal
plain = ~(us | eu)
out[plain] = pd.to_numeric(s[plain].str.replace(',', '.', regex=False), errors='coerce')

df['cold_rent'] = out
```

```
C:\Users\Perdorues\AppData\Local\Temp\ipykernel_15556\2430145821.py:7: UserWarning:
This pattern is interpreted as a regular expression, and has match groups. To actual
ly get the groups, use str.extract.
    us = s.str.contains(r'\d{3}(\D|$)') & s.str.contains(r'\.\d+$')    # 1,234.56
C:\Users\Perdorues\AppData\Local\Temp\ipykernel_15556\2430145821.py:8: UserWarning:
This pattern is interpreted as a regular expression, and has match groups. To actual
ly get the groups, use str.extract.
    eu = s.str.contains(r'\.\d{3}(\D|$)') & s.str.contains(r'\d+$')    # 1.234,56
```

```
In [92]: df["cold_rent"].sample(10)
```

```
Out[92]: 11988    1200.0
        2343    1900.0
        18987   1669.0
        15957   1296.6
        16171   1656.3
        21244   1299.0
        2880    540.0
        12924    325.0
        18799    999.0
        21813   1300.0
        Name: cold_rent, dtype: float64
```

```
In [93]: df['cold_rent'].describe()
```

```
Out[93]: count    15803.000000
        mean      1313.803160
        std       1188.053211
        min        1.095000
        25%        678.000000
        50%       1100.000000
        75%       1642.000000
        max       35000.000000
        Name: cold_rent, dtype: float64
```

The values seem reasonable, the outliers are expected, but nothing very extreme and unexpected. We will deal with outliers in the later stages.

## 2. Formatting 'area' feature

```
In [94]: df["area"].sample(5)
```

```
Out[94]: 16051    63 m²
        18925   50,8 m²
        16908   265 m²
        10696    79 m²
        10134    90 m²
        Name: area, dtype: object
```

```
In [95]: pattern = r'^\s*(?:\d{1,3}(?:[.]\d{3})+|\d+)(?:[.]\d+)?\s*m(?:^2|2)\s*$'
        mask = df['area'].astype(str).str.match(pattern)
        df_invalid = df[~mask]
        df_invalid['area'].sample(5)
```

```
Out[95]: 8460      100 m² Grundstück
20266     50,8 m² Grundstück
14695     1.100 m² Grundstück
8892      1.151 m² Grundstück
15853      175 m² Grundstück
Name: area, dtype: object
```

```
In [96]: # For some reason they have large values and I decided to also sample all the recor
df_invalid.sample(5)
```

```
Out[96]:
```

|              | listing_url                                                                                                       | cold_rent | area                   | rooms        | floor |
|--------------|-------------------------------------------------------------------------------------------------------------------|-----------|------------------------|--------------|-------|
| <b>22014</b> | <a href="https://www.immowelt.de/expose/4dda652c-42a4-4...">https://www.immowelt.de/expose/4dda652c-42a4-4...</a> | 1656.47   | 93,9 m²<br>Grundstück  | 3<br>Zimmer  | NaN   |
| <b>8952</b>  | <a href="https://www.immowelt.de/expose/9c681a48-0ac6-4...">https://www.immowelt.de/expose/9c681a48-0ac6-4...</a> | 2480.00   | 630 m²<br>Grundstück   | 3<br>Zimmer  | 2.0   |
| <b>13648</b> | <a href="https://www.immowelt.de/expose/92155f8c-ee7c-4...">https://www.immowelt.de/expose/92155f8c-ee7c-4...</a> | 2488.68   | 188 m²<br>Grundstück   | 5<br>Zimmer  | 4.0   |
| <b>19010</b> | <a href="https://www.immowelt.de/expose/2605b650-de83-4...">https://www.immowelt.de/expose/2605b650-de83-4...</a> | 5000.00   | 520 m²<br>Grundstück   | 10<br>Zimmer | 8.0   |
| <b>2851</b>  | <a href="https://www.immowelt.de/expose/748e7cef-82f0-4...">https://www.immowelt.de/expose/748e7cef-82f0-4...</a> | 2950.00   | 5.251 m²<br>Grundstück | 5<br>Zimmer  | 1.0   |

5 rows × 24 columns



The prices justify the big values of area. It just happend to be that for large areas the formatting doesn't match because of additional words like 'Grundstück'.

```
In [97]: mask = df['area'].astype(str).str.contains(',')
df_invalid = df[mask]
df_invalid['area'].sample(5)
```

```
Out[97]: 18591     71,5 m²
16619     38,8 m²
9826      67,7 m²
4898      36,7 m²
21769     37,7 m²
Name: area, dtype: object
```

```
In [98]: mask = df['area'].astype(str).str.contains(r'\.')
```

```
Out[98]: 15910    3.750 m² Grundstück
         19258    2.584 m² Grundstück
         10283    1.151 m² Grundstück
         2851     5.251 m² Grundstück
         22069         4.750 m²
         Name: area, dtype: object
```

```
In [99]: s = df['area'].astype(str).str.strip()

# Keep only the numeric part before the unit or before text.
s = s.str.replace(r'\s*(m²|m2)\b.*$', '', regex=True)

# Extract the first "number-like" token
num = s.str.extract(r'(\d+(?:[.,]\d+)*)')[0]

# Decide if it's "thousands grouping" like 3.750 or 1,234,567
# (one or more groups of exactly 3 digits after separators)
is_thousands = num.str.match(r'^\d{1,3}(?:[.,]\d{3})+$', na=False)

out = pd.Series(index=df.index, dtype='float64')

# Thousands-style: remove separators entirely -> integer
out[is_thousands] = pd.to_numeric(
    num[is_thousands].str.replace(r'[.,]', '', regex=True),
    errors='coerce'
)

# Non-thousands-style: treat comma as decimal, keep dot as decimal
out[~is_thousands] = pd.to_numeric(
    num[~is_thousands].str.replace(',', '.', regex=False),
    errors='coerce'
)

out.sample(5)
```

```
Out[99]: 12090    100.0
         9199     50.0
         13815    62.0
         21298    49.7
         223     100.0
         dtype: float64
```

```
In [100... # Format values look good so we save the changes.
df['area'] = out
df['area'].describe()
```

```
Out[100... count    15803.000000
mean       82.181301
std        141.620615
min         1.000000
25%         50.800000
50%         66.000000
75%         87.000000
max        5251.000000
Name: area, dtype: float64
```

No matter how careful we are with the formatting, outliers will always be present. We cannot avoid them.

### 3. Formatting 'rooms' feature

```
In [101... df["rooms"].unique()
```

```
Out[101... array(['2 Zimmer', '1 Zimmer', '4 Zimmer', '5 Zimmer', '3 Zimmer',  
      '1,5 Zimmer', '3,5 Zimmer', '8 Zimmer', '7 Zimmer', '2,5 Zimmer',  
      '6 Zimmer', '5,5 Zimmer', '4,5 Zimmer', '35 Zimmer', '2,2 Zimmer',  
      '6,5 Zimmer', '2,9 Zimmer', '10 Zimmer', '22 Zimmer'], dtype=object)
```

Most values seem legitimate, except the decimal ones. First let's remove the 'Zimmer' part and then have a detailed look into the unusual values with decimeters to get more information about them in order to decide how to deal with them.

```
In [102... df['rooms'] = df['rooms'].astype(str).str.strip().replace(r'\s*(Zimmer)\b.*$', '',  
  
# and convert to floating numbers  
df["rooms"] = (  
    df["rooms"]  
        .str.replace(",", ".", regex=False)    # turn 3,5 → 3.5  
        .astype(float)  
  
)  
  
df["rooms"].unique()
```

```
Out[102... array([ 2. ,  1. ,  4. ,  5. ,  3. ,  1.5,  3.5,  8. ,  7. ,  2.5,  6. ,  
      5.5,  4.5, 35. ,  2.2,  6.5,  2.9, 10. , 22. ])
```

```
In [103... mask = df['rooms'] % 1 != 0  
df_invalid = df[mask]  
df_invalid.sample(5)
```

Out[103...

|       | listing_url                                                                                                       | cold_rent | area  | rooms | floor |        |
|-------|-------------------------------------------------------------------------------------------------------------------|-----------|-------|-------|-------|--------|
| 18076 | <a href="https://www.immowelt.de/expose/36c923b2-3890-4...">https://www.immowelt.de/expose/36c923b2-3890-4...</a> | 790.58    | 72.5  | 2.5   | 5.0   | not_in |
| 13978 | <a href="https://www.immowelt.de/expose/5416d05e-4a50-4...">https://www.immowelt.de/expose/5416d05e-4a50-4...</a> | 355.00    | 50.0  | 1.5   | 4.0   |        |
| 18879 | <a href="https://www.immowelt.de/expose/42722074-2923-4...">https://www.immowelt.de/expose/42722074-2923-4...</a> | 1760.00   | 82.0  | 3.5   | 7.0   | in     |
| 21151 | <a href="https://www.immowelt.de/expose/217175d2-b19f-4...">https://www.immowelt.de/expose/217175d2-b19f-4...</a> | 1250.00   | 76.0  | 3.5   | NaN   | not_in |
| 9488  | <a href="https://www.immowelt.de/expose/b5fcf6f2-1d74-4...">https://www.immowelt.de/expose/b5fcf6f2-1d74-4...</a> | 1877.00   | 105.0 | 4.5   | 2.0   |        |

5 rows × 24 columns



The values seem legitimate. Adding 0.5 values, they might advertise small extra spaces. These definitely have their effect on the price, so rounding the values up or down would alter the real information.

I would still like to have a closer look at these two specific values, '2,9' and '2.2', in case they meant 29 rooms and 22 rooms.

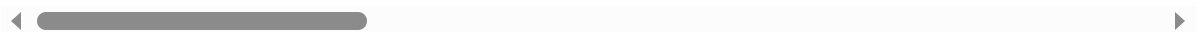
In [104...

```
df[(df["rooms"] == 2.2) | (df["rooms"] == 2.9)]
```

Out[104...

|       | listing_url                                       | cold_rent | area | rooms | floor |        |
|-------|---------------------------------------------------|-----------|------|-------|-------|--------|
| 21434 | https://www.immowelt.de/expose/9d2b7253-4684-4... | 1334.1    | 95.3 | 2.2   | NaN   | not_im |
| 19323 | https://www.immowelt.de/expose/9d2b7253-4684-4... | 1334.1    | 95.3 | 2.2   | 10.0  | not_im |
| 19087 | https://www.immowelt.de/expose/73f72612-5c96-4... | 1799.0    | 94.0 | 2.9   | 8.0   | not_im |
| 11593 | https://www.immowelt.de/expose/73f72612-5c96-4... | 1799.0    | 94.0 | 2.9   | 3.0   | not_im |
| 7398  | https://www.immowelt.de/expose/73f72612-5c96-4... | 1899.0    | 94.0 | 2.9   | 2.0   | not_im |
| 5460  | https://www.immowelt.de/expose/73f72612-5c96-4... | 1899.0    | 94.0 | 2.9   | 1.0   | not_im |
| 141   | https://www.immowelt.de/expose/73f72612-5c96-4... | 1899.0    | 94.0 | 2.9   | 0.0   | not_im |

7 rows × 24 columns



In [105... *# It seems like they made some types; Wanting to write 2,0 instead of 2,2 and 2,0 i*  
*# So we just round them to 2.*

In [106... `df.loc[(df["rooms"] == 2.2) | (df["rooms"] == 2.9), "rooms"] = 2`

In [107... `df["rooms"].unique()`

Out[107... `array([ 2. , 1. , 4. , 5. , 3. , 1.5, 3.5, 8. , 7. , 2.5, 6. ,  
5.5, 4.5, 35. , 6.5, 10. , 22. ])`

#### 4. Formatting 'flooring\_type' feature



```
In [108... df["flooring_type"].unique()
```

```
Out[108... array(['Unknown', 'fliesen', parkett', 'fliesen', 'fliesen, laminat',  
      'fliesen, holzdielen', 'holzdielen',  
      'fliesen, parkett, kunststoff', 'laminat, kunststoff, teppich',  
      'parkett', 'laminat', 'laminat, kunststoff', 'linoleum',  
      'parkett, laminat', 'stein', 'kunststoff',  
      'fliesen, holzdielen, parkett', 'fliesen, kunststoff',  
      'holzdielen, parkett, stein',  
      'fliesen, parkett, kunststoff, teppich',  
      'fliesen, holzdielen, parkett, stein', 'fliesen, parkett, laminat',  
      'parkett, stein', 'fliesen, parkett, laminat, teppich',  
      'holzdielen, parkett', 'fliesen, laminat, kunststoff',  
      'fliesen, parkett, stein', 'fliesen, parkett, teppich', 'estrich',  
      'fliesen, holzdielen, kunststoff', 'fliesen, marmor, parkett',  
      'holzdielen, laminat', 'fliesen, estrich, granit',  
      'marmor, parkett', 'parkett, doppelboden',  
      'fliesen, laminat, stein, teppich', 'fliesen, linoleum, teppich',  
      'laminat, kunststoff, linoleum', 'kunststoff, linoleum',  
      'fliesen, parkett, granit', 'holzdielen, stein',  
      'parkett, laminat, stein', 'parkett, kunststoff',  
      'fliesen, laminat, stein', 'holzdielen, parkett, laminat',  
      'parkett, laminat, kunststoff', 'teppich',  
      'fliesen, holzdielen, laminat',  
      'fliesen, holzdielen, parkett, laminat, kunststoff',  
      'holzdielen, kunststoff', 'marmor, holzdielen, stein',  
      'holzdielen, terrakotta', 'fliesen, holzdielen, terrakotta',  
      'fliesen, laminat, teppich',  
      'fliesen, holzdielen, parkett, kunststoff',  
      'fliesen, holzdielen, laminat, kunststoff',  
      'laminat, kunststoff, stein',  
      'fliesen, laminat, kunststoff, stein', 'fliesen, linoleum',  
      'fliesen, parkett, laminat, kunststoff', 'holzdielen, teppich',  
      'fliesen, teppich', 'laminat, stein', 'kunststoff, stein',  
      'fliesen, holzdielen, kunststoff, linoleum', 'marmor, laminat',  
      'marmor, holzdielen', 'holzdielen, parkett, terrakotta',  
      'parkett, terrakotta', 'holzdielen, laminat, stein',  
      'fliesen, laminat, terrakotta'], dtype=object)
```

We have so many types of values and some of them repeat under slightly different names. Also, we want them in english. We will try to map them in as few categories as possible while trying not to lose too much information.

We decided to map them into 8 pure categories since some of them have more, we will end up having more than 8 categories, but that is not avoidable without losing information.

```
In [109... floor_mapping = {  
    # Tiles  
    "fliesen": "Tiles",  
  
    # Wood  
    "parkett": "Wood",  
    "holzdielen": "Wood",  
    "doppelboden": "Wood",
```

```

# Laminate
"laminat": "Laminate",

# Vinyl / plastic family
"kunststoff": "Vinyl",
"linoleum": "Vinyl",

# Carpet
"teppich": "Carpet",

# Stone family
"stein": "Stone",
"marmor": "Stone",
"granit": "Stone",
"terrakotta": "Stone",

# Concrete/raw
"estrich": "Concrete",

# Unknown
"Unknown": "Unknown"
}

```

```

In [110... def map_flooring(value):
    if value == "Unknown":
        return "unknown"
    types = [t.strip() for t in value.split(",")]
    mapped = sorted(set(floor_mapping.get(t, t) for t in types))
    return ", ".join(mapped)

df["flooring_type"] = df["flooring_type"].apply(map_flooring)

```

```

In [111... df["flooring_type"].unique()

```

```

Out[111... array(['unknown', 'Tiles, Wood', 'Tiles', 'Laminate, Tiles', 'Wood',
      'Tiles, Vinyl, Wood', 'Carpet, Laminate, Vinyl', 'Laminate',
      'Laminate, Vinyl', 'Vinyl', 'Laminate, Wood', 'Stone',
      'Tiles, Vinyl', 'Stone, Wood', 'Carpet, Tiles, Vinyl, Wood',
      'Stone, Tiles, Wood', 'Laminate, Tiles, Wood',
      'Carpet, Laminate, Tiles, Wood', 'Laminate, Tiles, Vinyl',
      'Carpet, Tiles, Wood', 'Concrete', 'Concrete, Stone, Tiles',
      'Carpet, Laminate, Stone, Tiles', 'Carpet, Tiles, Vinyl',
      'Laminate, Stone, Wood', 'Vinyl, Wood', 'Laminate, Stone, Tiles',
      'Laminate, Vinyl, Wood', 'Carpet', 'Laminate, Tiles, Vinyl, Wood',
      'Carpet, Laminate, Tiles', 'Laminate, Stone, Vinyl',
      'Laminate, Stone, Tiles, Vinyl', 'Carpet, Wood', 'Carpet, Tiles',
      'Laminate, Stone', 'Stone, Vinyl'], dtype=object)

```

```

In [112... len(df["flooring_type"].unique())

```

```

Out[112... 37

```

When one-hot encoding the feature, 38 categories would be an overkill for our models. Let's try to reduce it more. That is why we will follow a slightly different approach,

where we rank the types of flooring from the most valuable to the least valuable in order to keep the most valuable type of flooring when there are multiple types of flooring listed.

```
In [113... floor_rank = {  
    "Stone": 5,  
    "Wood": 4,  
    "Tiles": 3,  
    "Laminate": 2,  
    "Vinyl": 2,  
    "Carpet": 1,  
    "Concrete": 0,  
    "unknown": -1  
}
```

```
In [114... def pick_most_expensive_floor(value):  
    if pd.isna(value):  
        return "unknown"  
  
    materials = [v.strip() for v in value.split(",")]  
  
    best_material = "unknown"  
    best_score = -1  
  
    for m in materials:  
        score = floor_rank.get(m, -1)  
        if score > best_score:  
            best_score = score  
            best_material = m  
  
    return best_material  
df["flooring_type"] = df["flooring_type"].apply(pick_most_expensive_floor)
```

```
In [115... df["flooring_type"].unique()
```

```
Out[115... array(['unknown', 'Wood', 'Tiles', 'Laminate', 'Vinyl', 'Stone',  
      'Concrete', 'Carpet'], dtype=object)
```

Now we have a reasonable number of categories.

## 6. Formatting 'energy\_source' feature

```
In [116... df["energy_source"].unique()
```

```
Out[116...] array(['Gas', 'Fernwärme', 'Gas, Solar, Luft-/Wasser-Wärmepumpe',
      'fossile Brennstoffe', 'Unknown', 'Luft-/Wasser-Wärmepumpe',
      'Erdwärme', 'Elektro, Luft-/Wasser-Wärmepumpe', 'Öl',
      'Gas, Blockheizkraftwerk',
      'Elektro, Solar, Luft-/Wasser-Wärmepumpe', 'Elektro, Gas',
      'Solar, Blockheizkraftwerk', 'Fernwärme, Gas', 'Gas, Solar',
      'Fernwärme, Öl', 'Elektro', 'Solar, Luft-/Wasser-Wärmepumpe',
      'Elektro, Solar', 'Gas, Luft-/Wasser-Wärmepumpe', 'Elektro, Öl',
      'Fernwärme, Blockheizkraftwerk',
      'Erdwärme, Solar, Luft-/Wasser-Wärmepumpe',
      'Fernwärme, Luft-/Wasser-Wärmepumpe', 'Gas, fossile Brennstoffe',
      'Erdwärme, Luft-/Wasser-Wärmepumpe', 'Blockheizkraftwerk',
      'Fernwärme, Elektro, Solar', 'Gas, Öl', 'Gas, Flüssiggas, Solar',
      'Pellets', 'Fernwärme, Gas, Blockheizkraftwerk', 'Solar',
      'Fernwärme, Erdwärme', 'Fernwärme, Elektro', 'Gas, Flüssiggas',
      'Gas,', 'Kohle, Gas, Öl', 'Elektro, Gas, Blockheizkraftwerk',
      'Flüssiggas', 'Fernwärme, Elektro, Blockheizkraftwerk',
      'Fernwärme, Elektro, Gas'], dtype=object)
```

We are facing the same problem. We have too many features and in german. We can follow a similar approach with the 'flooring\_type'.

```
In [117...] def classify_heating_rent(value):
    if pd.isna(value) or value == "Unknown":
        return "Unknown"

    types = [t.strip() for t in value.split(",") if t.strip()]

    has_district = "Fernwärme" in types
    has_gas = "Gas" in types
    has_oil = "Öl" in types or "Kohle" in types
    has_heatpump = "Luft-/Wasser-Wärmepumpe" in types or "Erdwärme" in types
    has_electric = "Elektro" in types
    has_solar = "Solar" in types
    has_chp = "Blockheizkraftwerk" in types

    # District is its own strong category
    if has_district and len(types) == 1:
        return "District Heating"

    # Modern systems
    if has_heatpump:
        if len(types) > 1:
            return "Hybrid"
        return "Heat Pump / Geothermal"

    # Gas baseline
    if has_gas and not has_oil:
        if len(types) > 1:
            return "Hybrid"
        return "Gas"

    # Oil / Coal
    if has_oil:
        return "Oil/Coal"
```

```

# Electric only
if has_electric and len(types) == 1:
    return "Electric"

# Mixed systems
if len(types) > 1:
    return "Hybrid"

return "Other"

```

```

In [118...] df["energy_source"] = df["energy_source"].apply(classify_heating_rent)

df["energy_source"].unique()

```

```

Out[118...] array(['Gas', 'District Heating', 'Hybrid', 'Other', 'Unknown',
        'Heat Pump / Geothermal', 'Oil/Coal', 'Electric'], dtype=object)

```

## 7. Formatting 'heating\_type' feature

```

In [119...] df["heating_type"].unique()

```

```

Out[119...] array(['Unknown', 'Zentralheizung', 'Zentralheizung: Fußbodenheizung',
        'Fußbodenheizung', 'Zentralheizung: Heizkörper', 'Etagenheizung',
        'individuell', 'Etagenheizung: Fußbodenheizung',
        'individuell: Heizkörper', 'Ofen', 'individuell: Fußbodenheizung'],
        dtype=object)

```

```

In [120...] def classify_heating_system(value):
    if pd.isna(value) or value == "Unknown":
        return "Unknown"

    value = value.strip()

    # Premium
    if "Fußbodenheizung" in value:
        return "Underfloor Heating"

    # Central
    if value.startswith("Zentralheizung"):
        return "Central Heating"

    # Unit-level heating
    if value.startswith("Etagenheizung"):
        return "Unit Heating"

    # Stove
    if "Ofen" in value:
        return "Stove Heating"

    # Individual systems
    if value.startswith("individuell"):
        return "Individual"

    return "Other"

```

```
In [121...] df["heating_type"] = df["heating_type"].apply(classify_heating_system)
df["heating_type"].unique()
```

```
Out[121...] array(['Unknown', 'Central Heating', 'Underfloor Heating', 'Unit Heating',
      'Individual', 'Stove Heating'], dtype=object)
```

## 8. Formatting 'property\_condition' feature

```
In [122...] df["property_condition"].unique()
```

```
Out[122...] array(['Gepflegt', 'Neubau', 'neuwertig', 'Erstbezug',
      'renoviert / saniert', 'Saniert', 'Neubau, Erstbezug',
      'Altbau (bis 1945), neuwertig', 'Neubau, neuwertig',
      'Altbau (bis 1945), Gepflegt',
      'Altbau (bis 1945), renoviert / saniert', 'Neubau, Massivhaus',
      'renovierungsbedürftig / sanierungsbedürftig',
      'Neubau, renoviert / saniert', 'Massivhaus, renoviert / saniert',
      'Altbau (bis 1945)', 'Massivhaus', 'teilsaniert',
      'Neubau, Massivhaus, Gepflegt', 'Neubau, Gepflegt',
      'Massivhaus, Gepflegt', 'Massivhaus, neuwertig',
      'Massivhaus, Erstbezug', 'Fertighaus, renoviert / saniert',
      'Erstbezug nach Renovierung', 'Modernisiert',
      'Neubau, Erstbezug, renoviert / saniert',
      'Altbau (bis 1945), neuwertig, renoviert / saniert',
      'Erstbezug, renoviert / saniert', 'neuwertig, renoviert / saniert',
      'Altbau (bis 1945), Saniert, renoviert / saniert',
      'Gepflegt, renoviert / saniert',
      'Neubau, neuwertig, renoviert / saniert',
      'Erstbezug nach Renovierung, renoviert / saniert',
      ', renoviert / saniert', 'Saniert, renoviert / saniert', 'Unknown'],
      dtype=object)
```

```
In [123...] def classify_condition(value):
    if pd.isna(value) or value == "Unknown":
        return "Unknown"

    value = value.strip()

    # Highest Level (brand new)
    if "Erstbezug" in value or "Neubau" in value:
        return "New / First Occupancy"

    # Like new
    if "neuwertig" in value:
        return "Like New"

    # Fully renovated
    if "Saniert" in value or "renoviert / saniert" in value or "Modernisiert" in va
        return "Renovated"

    # Partially renovated
    if "teilsaniert" in value:
        return "Partially Renovated"

    # Needs renovation
```

```

if "renovierungsbedürftig" in value or "sanierungsbedürftig" in value:
    return "Needs Renovation"

# Old building
if "Altbau" in value:
    return "Old Building"

# Well maintained baseline
if "Gepflegt" in value:
    return "Well Maintained"

return "Other"

```

```

In [124...] df["property_condition"] = df["property_condition"].apply(classify_condition)
df["property_condition"].unique()

```

```

Out[124...] array(['Well Maintained', 'New / First Occupancy', 'Like New',
      'Renovated', 'Old Building', 'Needs Renovation', 'Other',
      'Partially Renovated', 'Unknown'], dtype=object)

```

## 9. Formatting 'location' feature

Location is a very sensitive feature due to the fact that it can be used to fully identify the record on its own. Therefore, it is very important to perform k-anonymity generalization so that in case the dataset is published, the data won't be sufficient to identify the listing it corresponds to. We plan to keep both the district name and ZIP code. The ZIP code itself is a quasi-identifier, but without any other quasi-identifiers, keeping it in our dataset won't be a concern.

It is very important to mention that the ZIP code and district name carry almost identical information, so keeping them both in our dataset is not the best choice, but we are only keeping the ZIP code for other potential geographical applications, in our analysis, model training and dataset publication, we will only use the district name.

```

In [125...] df["location"].sample(10)

Out[125...] 19811      Wrangelstraße 67A, Kreuzberg, Berlin (10997)
17445      Scharnweberstraße 2, Oberschöneweide, Berlin (...
22407      Spandau, Berlin (13581)
4863      Am Hamburger Bahnhof, Moabit, Berlin (10557)
18179      Kamminer Straße 4, Charlottenburg, Berlin (10589)
15409      Halensee, Berlin (10711)
2918      Französisch Buchholz, Berlin (13158)
7694      Königin-Luise-Straße 15, Dahlem, Berlin (14195)
20086      Steglitz, Berlin (12163)
8529      Spilstraße 8, Dahlem, Berlin (14195)
Name: location, dtype: object

```

First, let's check if we have any samples that are not in Berlin

```
In [126... mask_no_berlin = ~df["location"].str.contains("Berlin", case=False, na=False)

df_no_berlin = df.loc[mask_no_berlin, "location"]

df_no_berlin.head(20)
```

```
Out[126... 18804 Kreuzberg, (10115)
13320 Kreuzberg, (10115)
8520 Moabit, (10551)
8374 Kreuzberg, (10115)
4882 Kreuzberg, (10115)
4457 Moabit, (10551)
22571 Mahlsdorf (12623)
14774 Mahlsdorf (12623)
9671 Mahlsdorf (12623)
Name: location, dtype: object
```

All samples in our data set are in Berlin, these 9 just happen to not have the word 'Berlin' explicitly written on them.

By observing many samples, it was noticed that this feature has this composition: [street\_name + house number], [area], Berlin, [(ZIP Code)]

We have to decide what information we should extract from this feature. Extracting the street number as a separate feature would mean too many categories since Berlin is a huge city. Extracting the area and ZIP code into two different features would cause an almost perfect collinearity, since they contain very similar information.

```
In [127... # Remove word 'Berlin'
df["location_clean"] = df["location"].str.replace("Berlin", "", regex=True, case=False)

# Extract ZIP
df["zip_code"] = df["location_clean"].str.extract(r"\\(\\d{5})\\s*$")

# Extract text before the last comma => name of district
df["district"] = df["location_clean"].str.extract(r",\\s*([^,]+?)\\s*\\(\\d{5}\\)\\s*$")

# Clean whitespace
df["district"] = df["district"].str.strip()
```

```
In [128... print(len(df["zip_code"].unique()))
```

191

```
In [129... print(len(df["district"].unique()))
```

52

```
In [130... df["district"].unique()
```



```
Out[130...] array(['', nan, '/ Bohnsdorf', '- Treptow', 'Tempelhof', '/ Mitte',
      '/ Charlottenburg', '/ Kreuzberg', '/ Lichtenrade',
      '/ Wilmersdorf', '-Spandau', '/ Mahlsdorf', 'Friedrichshain',
      '-Prenzlauer Berg', '-Reinickendorf', '/ Weißensee',
      '/ Friedrichshain', '-Treptow', '-Wilmersdorf', 'Kreuzberg',
      '-Zehlendorf', 'Köpenick', '/ Nikolassee', 'Kaulsdorf',
      '/ Tiergarten', '-Köpenick', '/ Adlershof', '-Dahlem',
      '/ Wendenschloss', '/ Köpenick', '-Mitte', '-Kaulsdorf',
      '-Hakenfelde', '-Friedrichsfelde', '/ Prenzlauer Berg',
      '/ Lichtenberg', '-Grünau', '/ Schmargendorf', '-12161', '-10711',
      '-12105', '-10589', '-12109', '-12203', '-12207', '-12205',
      '-12247', '-10627', '-12159', '/ Niederschönhausen', '-Weißensee',
      '/ Kladow'], dtype=object)
```

The most suitable way to deal with the location feature is to clean the 'district' feature using information from the internet to map the zip codes to their corresponding areas and for the missing values, we map them specifically by inspecting them and concluding the place.

```
In [131...] # keep only words and numbers
df["district"] = df["district"].str.extract(r'([A-Za-zÀ-ÖØ-öø-ÿ0-9]+)', expand=False)
df["district"].unique()
```

```
Out[131...] array([nan, 'Bohnsdorf', 'Treptow', 'Tempelhof', 'Mitte',
      'Charlottenburg', 'Kreuzberg', 'Lichtenrade', 'Wilmersdorf',
      'Spandau', 'Mahlsdorf', 'Friedrichshain', 'Prenzlauer',
      'Reinickendorf', 'Weißensee', 'Zehlendorf', 'Köpenick',
      'Nikolassee', 'Kaulsdorf', 'Tiergarten', 'Adlershof', 'Dahlem',
      'Wendenschloss', 'Hakenfelde', 'Friedrichsfelde', 'Lichtenberg',
      'Grünau', 'Schmargendorf', '12161', '10711', '12105', '10589',
      '12109', '12203', '12207', '12205', '12247', '10627', '12159',
      'Niederschönhausen', 'Kladow'], dtype=object)
```

```
In [132...] print(len(df["district"].unique()))
```

41

Now we have to map the ZIP codes to the corresponding area

```
In [133...] zip_dict = {
    "12161": "Tempelhof-Schöneberg",
    "10711": "Charlottenburg-Wilmersdorf",
    "12105": "Mariendorf",
    "10589": "Charlottenburg",
    "12109": "Mariendorf",
    "12203": "Steglitz",
    "12207": "Lichterfelde",
    "12205": "Lichterfelde",
    "12247": "Steglitz",
    "10627": "Charlottenburg",
    "12159": "Friedenau",
}

def map_zip_text(text):
    if pd.isna(text):
```

```

        return text

    text = str(text)
    for zip_code, district in zip_dict.items():
        text = text.replace(zip_code, district)
    return text

df["district"] = df["district"].apply(map_zip_text)

```

In [134... df["district"].unique()

Out[134... array([nan, 'Bohnsdorf', 'Treptow', 'Tempelhof', 'Mitte',  
 'Charlottenburg', 'Kreuzberg', 'Lichtenrade', 'Wilmerdorf',  
 'Spandau', 'Mahlsdorf', 'Friedrichshain', 'Prenzlauer',  
 'Reinickendorf', 'Weißensee', 'Zehlendorf', 'Köpenick',  
 'Nikolassee', 'Kaulsdorf', 'Tiergarten', 'Adlershof', 'Dahlem',  
 'Wendenschloss', 'Hakenfelde', 'Friedrichsfelde', 'Lichtenberg',  
 'Grünau', 'Schmargendorf', 'Tempelhof-Schöneberg',  
 'Charlottenburg-Wilmerdorf', 'Mariendorf', 'Steglitz',  
 'Lichterfelde', 'Friedenau', 'Niederschönhausen', 'Kladow'],  
 dtype=object)

In [135... print(len(df["district"].unique()))

36

In [136... mask = df["district"].isna()  
 df[mask][['district', 'zip\_code', 'location']]

Out[136...

|       | district | zip_code | location                                          |
|-------|----------|----------|---------------------------------------------------|
| 22653 | NaN      | 10405    | Prenzlauer Berg, Berlin (10405)                   |
| 22652 | NaN      | 13347    | Reinickendorfer Str. 1, Gesundbrunnen, Berlin ... |
| 22651 | NaN      | 13629    | Heinrich-Hertz-Straße 40, Siemensstadt, Berlin... |
| 22642 | NaN      | 10117    | Mitte, Berlin (10117)                             |
| 22641 | NaN      | 10318    | Karlshorst, Berlin (10318)                        |
| ...   | ...      | ...      | ...                                               |
| 8     | NaN      | 10179    | Am Köllnischen Park, Mitte, Berlin (10179)        |
| 9     | NaN      | 13129    | Karower Damm, Blankenburg, Berlin (13129)         |
| 6     | NaN      | 10437    | Topsstraße, Prenzlauer Berg, Berlin (10437)       |
| 1     | NaN      | 12555    | Spreestraße, Köpenick, Berlin (12555)             |
| 3     | NaN      | 12555    | Bahnhofstraße, Köpenick, Berlin (12555)           |

15522 rows × 3 columns

All these need to be mapped

In [137...

```
zip_dict = {  
    # Charlottenburg-Wilmersdorf (106xx-107xx)  
    "10551": "Tiergarten",  
    "10553": "Tiergarten",  
    "10555": "Tiergarten",  
    "10557": "Tiergarten",  
    "10559": "Tiergarten",  
    "10585": "Charlottenburg",  
    "10587": "Charlottenburg",  
    "10589": "Charlottenburg",  
    "10623": "Charlottenburg",  
    "10625": "Charlottenburg",  
    "10627": "Charlottenburg",  
    "10629": "Charlottenburg",  
    "10707": "Wilmersdorf",  
    "10709": "Wilmersdorf",  
    "10711": "Wilmersdorf",  
    "10713": "Wilmersdorf",  
    "10715": "Wilmersdorf",  
    "10717": "Wilmersdorf",  
    "10719": "Wilmersdorf",  
    "10777": "Schöneberg",  
    "10779": "Schöneberg",  
    "10781": "Schöneberg",  
    "10783": "Schöneberg",  
    "10785": "Schöneberg",  
    "10787": "Schöneberg",  
    "10789": "Schöneberg",  
  
    # Tempelhof-Schöneberg (107xx, 108xx, 121xx)  
    "10823": "Schöneberg",  
    "10825": "Schöneberg",  
    "10827": "Schöneberg",  
    "10829": "Schöneberg",  
    "12043": "Neukölln",  
    "12045": "Neukölln",  
    "12047": "Neukölln",  
    "12049": "Neukölln",  
    "12051": "Neukölln",  
    "12053": "Neukölln",  
    "12055": "Neukölln",  
    "12057": "Neukölln",  
    "12059": "Neukölln",  
    "12099": "Tempelhof",  
    "12101": "Tempelhof",  
    "12103": "Tempelhof",  
    "12105": "Tempelhof",  
    "12107": "Tempelhof",  
    "12109": "Tempelhof",  
    "12157": "Steglitz",  
    "12159": "Friedenau",  
    "12161": "Friedenau",  
    "12163": "Steglitz",  
    "12165": "Steglitz",  
    "12167": "Steglitz",  
}
```

```
"12169": "Steglitz",

# Steglitz-Zehlendorf (121xx, 122xx, 141xx)
"12203": "Steglitz",
"12205": "Lichterfelde",
"12207": "Lichterfelde",
"12209": "Lichterfelde",
"12247": "Steglitz",
"12249": "Steglitz",
"12277": "Marienfelde",
"12279": "Marienfelde",
"12305": "Lichtenrade",
"12307": "Lichtenrade",
"12309": "Lichtenrade",
"12347": "Britz",
"12349": "Britz",
"12351": "Britz",
"12353": "Britz",
"12355": "Britz",
"12357": "Britz",
"12359": "Britz",

# Tempelhof-Schöneberg (124xx)
"12435": "Alt-Treptow",
"12437": "Alt-Treptow",
"12439": "Niederschöneweide",
"12459": "Oberschöneweide",
"12487": "Johannisthal",
"12489": "Johannisthal",

# Neukölln (120xx, 123xx)
"12524": "Altglienicke",
"12526": "Bohnsdorf",
"12527": "Grünau",
"12555": "Köpenick",
"12557": "Köpenick",
"12559": "Köpenick",
"12587": "Friedrichshagen",
"12589": "Rahnsdorf",

# Treptow-Köpenick (124xx, 125xx)
"12619": "Hellersdorf",
"12621": "Hellersdorf",
"12623": "Mahlsdorf",
"12627": "Hellersdorf",
"12629": "Hellersdorf",
"12679": "Marzahn",
"12681": "Marzahn",
"12683": "Biesdorf",
"12685": "Marzahn",
"12687": "Marzahn",
"12689": "Marzahn",

# Marzahn-Hellersdorf (126xx)
"13051": "Hohenschönhausen",
"13053": "Hohenschönhausen",
```

"13055": "Hohenschönhausen",  
"13057": "Hohenschönhausen",  
"13059": "Hohenschönhausen",  
"13086": "Weißensee",  
"13088": "Weißensee",  
"13089": "Weißensee",

*# Lichtenberg (103xx, 130xx)*

"13125": "Buch",  
"13127": "Französisch Buchholz",  
"13129": "Blankenburg",  
"13156": "Niederschönhausen",  
"13158": "Niederschönhausen",  
"13159": "Niederschönhausen",  
"13187": "Pankow",  
"13189": "Pankow",

*# Pankow (101xx, 104xx, 131xx)*

"13347": "Wedding",  
"13349": "Wedding",  
"13351": "Wedding",  
"13353": "Wedding",  
"13355": "Gesundbrunnen",  
"13357": "Gesundbrunnen",  
"13359": "Gesundbrunnen",

*# Mitte (101xx, 133xx)*

"13403": "Reinickendorf",  
"13405": "Reinickendorf",  
"13407": "Reinickendorf",  
"13409": "Reinickendorf",  
"13435": "Wittenau",  
"13437": "Wittenau",  
"13439": "Wittenau",  
"13465": "Hermisdorf",  
"13467": "Hermisdorf",  
"13469": "Hermisdorf",

*# Reinickendorf (134xx, 135xx)*

"13503": "Tegel",  
"13505": "Tegel",  
"13507": "Tegel",  
"13509": "Tegel",  
"13581": "Spandau",  
"13583": "Spandau",  
"13585": "Spandau",  
"13587": "Spandau",  
"13589": "Spandau",  
"13591": "Staaken",  
"13593": "Staaken",  
"13595": "Staaken",  
"13597": "Spandau",  
"13599": "Spandau",

*# Spandau (135xx, 140xx)*

"14050": "Charlottenburg",

"14052": "Charlottenburg",  
"14055": "Charlottenburg",  
"14057": "Charlottenburg",  
"14059": "Charlottenburg",  
"14089": "Kladow",

*# Charlottenburg-Wilmersdorf (140xx)*

"14109": "Wannsee",  
"14129": "Nikolassee",  
"14163": "Zehlendorf",  
"14165": "Zehlendorf",  
"14167": "Zehlendorf",  
"14169": "Zehlendorf",  
"14193": "Grunewald",  
"14195": "Dahlem",  
"14197": "Wilmersdorf",  
"14199": "Wilmersdorf",

*# Steglitz-Zehlendorf (141xx)*

"10115": "Mitte",  
"10117": "Mitte",  
"10119": "Mitte",  
"10178": "Mitte",  
"10179": "Mitte",  
"10243": "Friedrichshain",  
"10245": "Friedrichshain",  
"10247": "Friedrichshain",  
"10249": "Friedrichshain",  
"10315": "Lichtenberg",  
"10317": "Lichtenberg",  
"10318": "Lichtenberg",  
"10319": "Lichtenberg",  
"10365": "Lichtenberg",  
"10367": "Lichtenberg",  
"10369": "Lichtenberg",  
"10405": "Prenzlauer Berg",  
"10407": "Prenzlauer Berg",  
"10409": "Prenzlauer Berg",  
"10435": "Prenzlauer Berg",  
"10437": "Prenzlauer Berg",  
"10439": "Prenzlauer Berg",  
"10551": "Tiergarten",  
"10553": "Tiergarten",  
"10555": "Tiergarten",  
"10557": "Tiergarten",  
"10559": "Tiergarten",  
"10585": "Charlottenburg",  
"10587": "Charlottenburg",  
"10589": "Charlottenburg",  
"10623": "Charlottenburg",  
"10625": "Charlottenburg",  
"10627": "Charlottenburg",  
"10629": "Charlottenburg",  
"10707": "Wilmersdorf",  
"10709": "Wilmersdorf",  
"10711": "Wilmersdorf",

"10713": "Wilmersdorf",  
"10715": "Wilmersdorf",  
"10717": "Wilmersdorf",  
"10719": "Wilmersdorf",  
"10777": "Schöneberg",  
"10779": "Schöneberg",  
"10781": "Schöneberg",  
"10783": "Schöneberg",  
"10785": "Schöneberg",  
"10787": "Schöneberg",  
"10789": "Schöneberg",  
"10799": "Schöneberg",  
"10823": "Schöneberg",  
"10825": "Schöneberg",  
"10827": "Schöneberg",  
"10829": "Schöneberg",  
"10961": "Kreuzberg",  
"10963": "Kreuzberg",  
"10965": "Kreuzberg",  
"10967": "Kreuzberg",  
"10969": "Kreuzberg",  
"10997": "Kreuzberg",  
"10999": "Kreuzberg",  
"12043": "Neukölln",  
"12045": "Neukölln",  
"12047": "Neukölln",  
"12049": "Neukölln",  
"12051": "Neukölln",  
"12053": "Neukölln",  
"12055": "Neukölln",  
"12057": "Neukölln",  
"12059": "Neukölln",  
"12099": "Tempelhof",  
"12101": "Tempelhof",  
"12103": "Tempelhof",  
"12105": "Tempelhof",  
"12107": "Tempelhof",  
"12109": "Tempelhof",  
"12157": "Steglitz",  
"12159": "Friedenau",  
"12161": "Friedenau",  
"12163": "Steglitz",  
"12165": "Steglitz",  
"12167": "Steglitz",  
"12169": "Steglitz",  
"12203": "Steglitz",  
"12205": "Lichterfelde",  
"12207": "Lichterfelde",  
"12209": "Lichterfelde",  
"12247": "Steglitz",  
"12249": "Steglitz",  
"12277": "Marienfelde",  
"12279": "Marienfelde",  
"12305": "Lichtenrade",  
"12307": "Lichtenrade",  
"12309": "Lichtenrade",

"12347": "Britz",  
"12349": "Britz",  
"12351": "Britz",  
"12353": "Britz",  
"12355": "Britz",  
"12357": "Britz",  
"12359": "Britz",  
"12435": "Alt-Treptow",  
"12437": "Alt-Treptow",  
"12439": "Niederschöneweide",  
"12459": "Oberschöneweide",  
"12487": "Johannisthal",  
"12489": "Johannisthal",  
"12524": "Altglienicke",  
"12526": "Bohnsdorf",  
"12527": "Grünau",  
"12555": "Köpenick",  
"12557": "Köpenick",  
"12559": "Köpenick",  
"12587": "Friedrichshagen",  
"12589": "Rahnsdorf",  
"12619": "Hellersdorf",  
"12621": "Hellersdorf",  
"12623": "Mahlsdorf",  
"12627": "Hellersdorf",  
"12629": "Hellersdorf",  
"12679": "Marzahn",  
"12681": "Marzahn",  
"12683": "Biesdorf",  
"12685": "Marzahn",  
"12687": "Marzahn",  
"12689": "Marzahn",  
"13051": "Hohenschönhausen",  
"13053": "Hohenschönhausen",  
"13055": "Hohenschönhausen",  
"13057": "Hohenschönhausen",  
"13059": "Hohenschönhausen",  
"13086": "Weißensee",  
"13088": "Weißensee",  
"13089": "Weißensee",  
"13125": "Buch",  
"13127": "Französisch Buchholz",  
"13129": "Blankenburg",  
"13156": "Niederschönhausen",  
"13158": "Niederschönhausen",  
"13159": "Niederschönhausen",  
"13187": "Pankow",  
"13189": "Pankow",  
"13347": "Wedding",  
"13349": "Wedding",  
"13351": "Wedding",  
"13353": "Wedding",  
"13355": "Gesundbrunnen",  
"13357": "Gesundbrunnen",  
"13359": "Gesundbrunnen",  
"13403": "Reinickendorf",



```

"13405": "Reinickendorf",
"13407": "Reinickendorf",
"13409": "Reinickendorf",
"13435": "Wittenau",
"13437": "Wittenau",
"13439": "Wittenau",
"13465": "Hermisdorf",
"13467": "Hermisdorf",
"13469": "Hermisdorf",
"13503": "Tegel",
"13505": "Tegel",
"13507": "Tegel",
"13509": "Tegel",
"13581": "Spandau",
"13583": "Spandau",
"13585": "Spandau",
"13587": "Spandau",
"13589": "Spandau",
"13591": "Staaken",
"13593": "Staaken",
"13595": "Staaken",
"13597": "Spandau",
"13599": "Spandau",
"13627": "Spandau",
"13629": "Siemensstadt",
"14050": "Charlottenburg",
"14052": "Charlottenburg",
"14055": "Charlottenburg",
"14057": "Charlottenburg",
"14059": "Charlottenburg",
"14089": "Kladow",
"14109": "Wannsee",
"14129": "Nikolassee",
"14163": "Zehlendorf",
"14165": "Zehlendorf",
"14167": "Zehlendorf",
"14169": "Zehlendorf",
"14193": "Grunewald",
"14195": "Dahlem",
"14197": "Wilmerdorf",
"14199": "Wilmerdorf"
}

```

In [138...

```

def map_zip_text(row):
    # Check if area is NaN and zip code exists
    if pd.isna(row['district']) and pd.notna(row['zip_code']):
        zip_code = str(row['zip_code']).strip()

        if zip_code in zip_dict:
            return f"{zip_dict[zip_code]}"

        return row['district']

# Apply the function row-wise
df["district"] = df.apply(map_zip_text, axis=1)

```

In [139...

```
df["district"].unique()
```

Out[139...

```
array(['Prenzlauer Berg', 'Wedding', 'Siemensstadt', 'Mitte',
      'Lichtenberg', 'Weißensee', 'Charlottenburg', 'Wilmerdorf',
      'Bohnsdorf', 'Friedrichshain', 'Tempelhof', 'Schöneberg',
      'Französisch Buchholz', 'Tegel', 'Kreuzberg', 'Friedenau',
      'Zehlendorf', 'Britz', 'Friedrichshagen', 'Köpenick', 'Mahlisdorf',
      'Treptow', 'Neukölln', 'Grünau', 'Gesundbrunnen', 'Staaken',
      'Pankow', 'Steglitz', 'Reinickendorf', 'Tiergarten', 'Lichtenrade',
      'Johannisthal', 'Kladow', 'Biesdorf', 'Hohenschönhausen',
      'Lichterfelde', 'Niederschöneweide', 'Oberschöneweide',
      'Blankenburg', 'Grunewald', 'Wannsee', 'Dahlem', 'Hermsdorf',
      'Spandau', 'Altglienicke', 'Niederschönhausen', 'Hellersdorf',
      'Alt-Treptow', 'Prenzlauer', 'Marzahn', 'Marienfelde', 'Wittenau',
      'Buch', 'Nikolassee', 'Rahnsdorf', 'Kaulsdorf', 'Adlershof',
      'Wendenschloss', 'Hakenfelde', 'Friedrichsfelde', 'Schmargendorf',
      'Tempelhof-Schöneberg', 'Charlottenburg-Wilmerdorf', 'Mariendorf',
      nan], dtype=object)
```

In [140...

```
mask = df["district"].isna()
df[mask][['district', 'zip_code', 'location']]
```

Out[140...

|       | district | zip_code | location                            |
|-------|----------|----------|-------------------------------------|
| 13752 | NaN      | NaN      | Prenzlauer Berg, Berlin (100001199) |
| 10778 | NaN      | NaN      | Wilmerdorf, Berlin (10)             |
| 10570 | NaN      | NaN      | Niederschönhausen, Berlin (13)      |
| 9311  | NaN      | NaN      | Prenzlauer Berg, Berlin (100001199) |
| 7359  | NaN      | NaN      | Französisch Buchholz, Berlin (13)   |
| 7321  | NaN      | NaN      | Wilmerdorf, Berlin (10)             |
| 2598  | NaN      | NaN      | Prenzlauer Berg, Berlin (100001199) |
| 2159  | NaN      | NaN      | Niederschönhausen, Berlin (13)      |
| 1431  | NaN      | NaN      | Friedrichshain, Berlin (10)         |
| 1327  | NaN      | NaN      | Berlin (13)                         |
| 1068  | NaN      | NaN      | Wilmerdorf, Berlin (10)             |
| 879   | NaN      | NaN      | Prenzlauer Berg, Berlin (10)        |
| 625   | NaN      | NaN      | Schöneberg, Berlin (100001199)      |
| 387   | NaN      | NaN      | Französisch Buchholz, Berlin (13)   |

There are still nan values left. These are all valid samples except sample number 1327 which we can simply drop.

In [141...

```
df = df.drop(index=1327)
```

```
In [142... # Since we dont have many samples we edit them manually.
df.loc[[13752, 9311, 2598, 879], 'district'] = 'Prenzlauer Berg'
df.loc[[10778, 7321, 1068], 'district'] = 'Wilmerdorf'
df.loc[[10570, 2159], 'district'] = 'Niederschönhausen'
df.loc[[7359, 387], 'district'] = 'Französisch Buchholz'
df.loc[1431, 'district'] = 'Friedrichshain'
df.loc[625, 'district'] = 'Schöneberg'
```

```
In [143... df[df['district'].isna()][['location', 'district']]
```

```
Out[143... location district
```

```
In [144... len(df["district"].unique())
```

```
Out[144... 64
```

We managed to clean all the values, but we still have a problem, that being having 64 categories (districts). Maybe we have some values that we can cluster together.

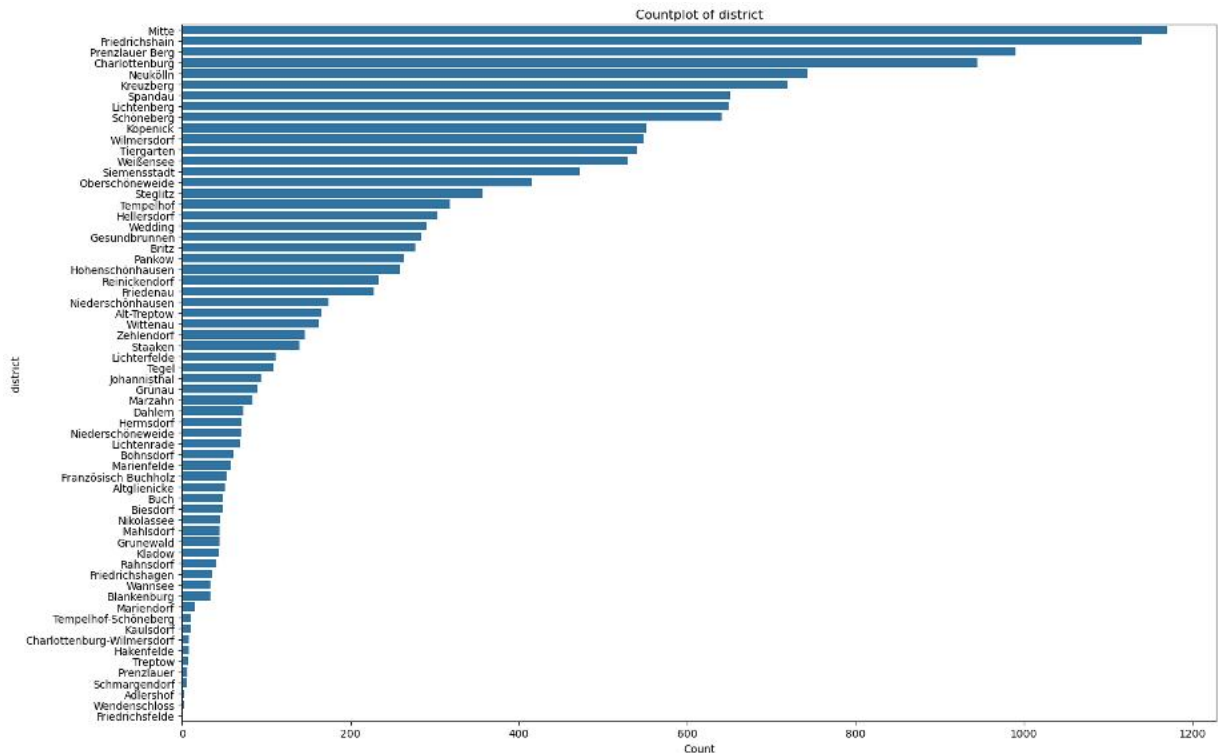
```
In [145... len(df[df['zip_code'].isna()])
```

```
Out[145... 13
```

We have only 13 missing records with the ZIP code missing, so it won't cost us anything to just drop them.

```
In [146... df = df.dropna(subset=["zip_code"])
```

```
In [147... plt.figure(figsize=(16, 10))
sns.countplot(data=df, y="district", order=df["district"].value_counts().index)
plt.title('Countplot of district')
plt.xlabel('Count')
plt.tight_layout()
plt.show()
```

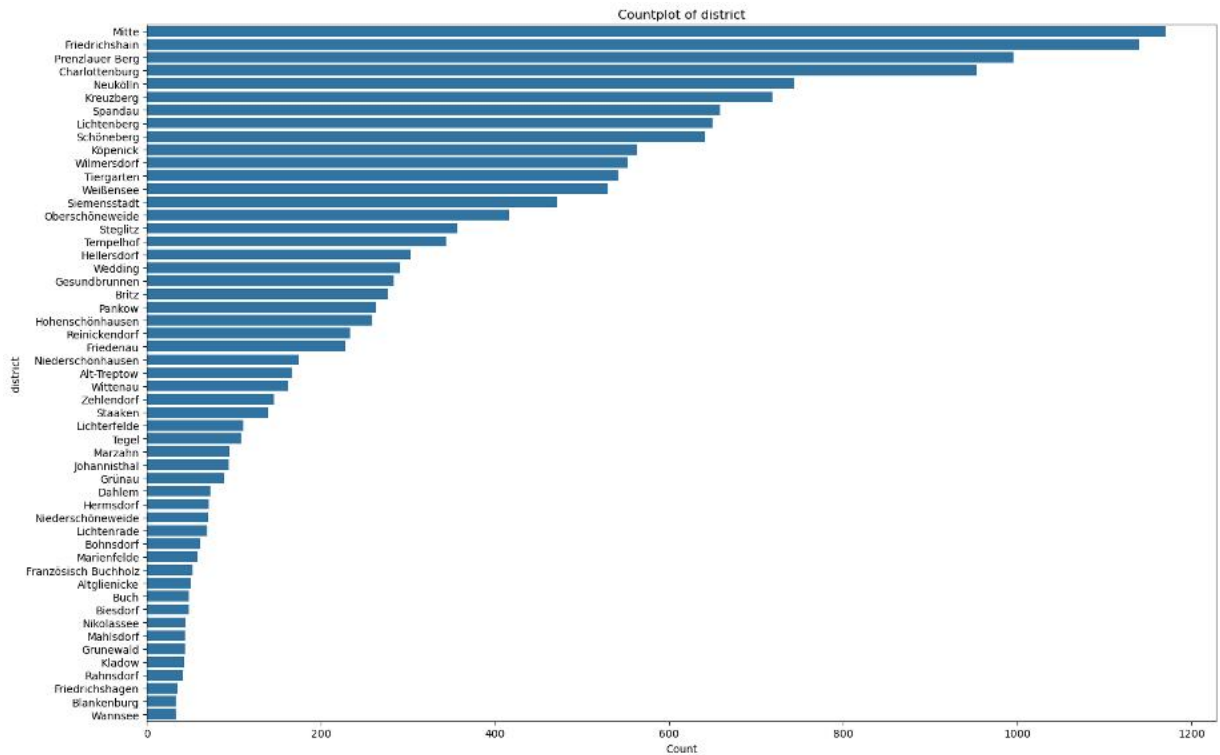


```
In [148... # We have 'Prenzlauer Berg' written also as 'Prenzlauer', so we can just merge the
# df[df['district'] == 'Prenzlauer']['district']
df.loc[df['district'] == 'Prenzlauer', 'district'] = 'Prenzlauer Berg'
```

```
In [149... # We should also group categories with very few occurances into more general district
df.loc[df['district'] == 'Friedrichsfelde', 'district'] = 'Lichtenberg'
df.loc[df['district'] == 'Wendenschloss', 'district'] = 'Köpenick'
df.loc[df['district'] == 'Adlershof', 'district'] = 'Köpenick'
df.loc[df['district'] == 'Schmargendorf', 'district'] = 'Wilmsdorf'
df.loc[df['district'] == 'Treptow', 'district'] = 'Köpenick'
df.loc[df['district'] == 'Hakenfelde', 'district'] = 'Spandau'
df.loc[df['district'] == 'Charlottenburg-Wilmersdorf', 'district'] = 'Charlottenburg'
df.loc[df['district'] == 'Kaulsdorf', 'district'] = 'Marzahn'
df.loc[df['district'] == 'Tempelhof-Schöneberg', 'district'] = 'Tempelhof'
df.loc[df['district'] == 'Mariendorf', 'district'] = 'Tempelhof' # 15 values

# As noticed some arbitrary decisions were made, like converting 'Tempelhof-Schöneberg'
# instead of 'Schöneberg' to balance the categories more.
# Also this impact is relevant since they were less than 15 samples.
```

```
In [150... plt.figure(figsize=(16, 10))
sns.countplot(data=df, y="district", order=df["district"].value_counts().index)
plt.title('Countplot of district')
plt.xlabel('Count')
plt.tight_layout()
plt.show()
```



```
In [151...] len(df["district"].unique())
```

```
Out[151...] 53
```

```
In [152...] len(df[df["district"] == "Blankenburg"])
```

```
Out[152...] 33
```

```
In [153...] df.shape
```

```
Out[153...] (15789, 27)
```

The categories with the lowest number of samples are Wannsee and Blankenburg with 33 samples, which is not very low. We have 53 categories, which is quite a lot for a feature to have, but the type of the feature justifies it. I would prefer not to cluster any more categories together to preserve information. There are many ML models out there that work well with features with this many categories. A ratio of 53 categories to 15,000 samples is also a good ratio.

## Part 4: Handling outliers

### 1. Handling 'cold\_rent' feature

```
In [154...] df['cold_rent'].describe()
```

```
Out[154... count    15789.000000
mean      1314.197374
std        1188.497966
min         1.095000
25%         677.000000
50%        1100.000000
75%        1642.000000
max        35000.000000
Name: cold_rent, dtype: float64
```

We clearly have outliers and a lot of them. We have to find a way to deal with them. But first we need to inspect them in detail to see whether they are a type or just an invalid sample.

```
In [155... # Knowing Berlin anything below 300 euros is not possible.
len(df[df['cold_rent'] < 300])
```

```
Out[155... 163
```

```
In [156... # We have to inspect at least 163 values. Every sample is valuable and should not b
# to inspect them one by one.
# We will start with a threshold of '2' and increase it gradually until we reach 30
df[df['cold_rent'] < 2]
```

Out[156...

|       | listing_url                                                                                                       | cold_rent | area | rooms | floor |        |
|-------|-------------------------------------------------------------------------------------------------------------------|-----------|------|-------|-------|--------|
| 28    | <a href="https://www.immowelt.de/expose/ed2ff766-9412-4...">https://www.immowelt.de/expose/ed2ff766-9412-4...</a> | 1.349     | 65.8 | 2.0   | 0.0   | im     |
| 26    | <a href="https://www.immowelt.de/expose/7a4aec64-ab72-4...">https://www.immowelt.de/expose/7a4aec64-ab72-4...</a> | 1.449     | 66.5 | 2.0   | 0.0   | not_im |
| 15    | <a href="https://www.immowelt.de/expose/d4043120-352d-4...">https://www.immowelt.de/expose/d4043120-352d-4...</a> | 1.095     | 45.0 | 2.0   | 0.0   | im     |
| 17    | <a href="https://www.immowelt.de/expose/0d680cb5-7a9c-4...">https://www.immowelt.de/expose/0d680cb5-7a9c-4...</a> | 1.595     | 67.5 | 2.0   | 0.0   | im     |
| 14    | <a href="https://www.immowelt.de/expose/94b19a7d-b5d4-4...">https://www.immowelt.de/expose/94b19a7d-b5d4-4...</a> | 1.095     | 40.0 | 2.0   | 0.0   | im     |
| 4     | <a href="https://www.immowelt.de/expose/03a3d0b6-9fab-4...">https://www.immowelt.de/expose/03a3d0b6-9fab-4...</a> | 1.399     | 65.8 | 2.0   | -1.0  | not_im |
| 0     | <a href="https://www.immowelt.de/expose/f6e9dd76-dbe1-4...">https://www.immowelt.de/expose/f6e9dd76-dbe1-4...</a> | 1.499     | 58.3 | 2.0   | -1.0  | not_im |
| 22169 | <a href="https://www.immowelt.de/expose/f899bf21-d112-4...">https://www.immowelt.de/expose/f899bf21-d112-4...</a> | 1.250     | 80.0 | 3.0   | NaN   |        |
| 18694 | <a href="https://www.immowelt.de/expose/f899bf21-d112-4...">https://www.immowelt.de/expose/f899bf21-d112-4...</a> | 1.250     | 80.0 | 3.0   | 6.0   |        |
| 14493 | <a href="https://www.immowelt.de/expose/f899bf21-d112-4...">https://www.immowelt.de/expose/f899bf21-d112-4...</a> | 1.250     | 80.0 | 3.0   | 4.0   |        |
| 13267 | <a href="https://www.immowelt.de/expose/f899bf21-d112-4...">https://www.immowelt.de/expose/f899bf21-d112-4...</a> | 1.250     | 80.0 | 3.0   | 3.0   |        |
| 29    | <a href="https://www.immowelt.de/expose/a7088184-d5a8-4...">https://www.immowelt.de/expose/a7088184-d5a8-4...</a> | 1.530     | 61.1 | 3.0   | 0.0   | not_im |
| 12    | <a href="https://www.immowelt.de/expose/1a0ba5cb-3931-4...">https://www.immowelt.de/expose/1a0ba5cb-3931-4...</a> | 1.150     | 40.9 | 2.0   | 0.0   | not_im |

|   | listing_url                                                                                                       | cold_rent | area | rooms | floor |        |
|---|-------------------------------------------------------------------------------------------------------------------|-----------|------|-------|-------|--------|
| 8 | <a href="https://www.immowelt.de/expose/aba79292-351c-4...">https://www.immowelt.de/expose/aba79292-351c-4...</a> | 1.330     | 49.3 | 2.0   | -1.0  | not_im |
| 6 | <a href="https://www.immowelt.de/expose/49ae5e0a-6f97-4...">https://www.immowelt.de/expose/49ae5e0a-6f97-4...</a> | 1.645     | 85.8 | 3.0   | -1.0  | not_im |
| 1 | <a href="https://www.immowelt.de/expose/56cf72dd-00ea-4...">https://www.immowelt.de/expose/56cf72dd-00ea-4...</a> | 1.178     | 45.7 | 2.0   | -1.0  | not_im |

16 rows × 27 columns

For this one the problem and solution are obvious, we just have to multiple the values by 1000.

```
In [157... df.loc[df["cold_rent"] < 2, "cold_rent"] *= 1000
```

```
In [158... df[df['cold_rent'] < 5]
```



Out[158...

|     | listing_url                                       | cold_rent | area  | rooms | floor | fr       |
|-----|---------------------------------------------------|-----------|-------|-------|-------|----------|
| 22  | https://www.immowelt.de/expose/d5dbcc47-60a6-4... | 2.300     | 115.0 | 3.0   | 0.0   | u        |
| 18  | https://www.immowelt.de/expose/94a3c69c-14a9-4... | 2.040     | 88.6  | 3.0   | 0.0   | immr     |
| 13  | https://www.immowelt.de/expose/25d8e8ff-8de6-4... | 2.110     | 87.8  | 3.0   | 0.0   | immr     |
| 7   | https://www.immowelt.de/expose/7e269628-a21a-4... | 2.249     | 85.8  | 3.0   | -1.0  | not_immr |
| 778 | https://www.immowelt.de/expose/00aa6fd9-a5ab-4... | 3.000     | 50.0  | 2.0   | 0.0   | u        |

5 rows × 27 columns



In [159...

```
# the last one looks weird I can't tell if it is 3000 or 300. Let's just inspect th
with pd.option_context('display.max_colwidth', None):
    print(df.loc[778]['listing_url'])
```

https://www.immowelt.de/expose/00aa6fd9-a5ab-4533-9d8f-f853047e4aa0?serp\_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D149#ln=classified\_search\_results&m=classified\_search\_results\_classified\_classified\_detail\_L

In [160...

```
# even after inspecting the sample, I cannot tell and I don't want to risk making a
# I will simply delete the sample.
df = df.drop(index=778)
```

In [161...

```
df.loc[df["cold_rent"] < 5, "cold_rent"] *= 1000
```

In [162...

```
df[df['cold_rent'] < 5]
```

Out[162...

| listing_url | cold_rent | area | rooms | floor | free_from | location | has_balkon | has_terra |
|-------------|-----------|------|-------|-------|-----------|----------|------------|-----------|
|-------------|-----------|------|-------|-------|-----------|----------|------------|-----------|

0 rows × 27 columns



```

In [163... # Let's inspect the ones with small values
with pd.option_context('display.max_colwidth', None):
    print(df.loc[[11, 2]]['listing_url'])

11    https://www.immowelt.de/expose/2605b650-de83-4aeb-8bc8-bc8966e1d8a0?serp_view=
list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3
DAD08DE8634#ln=classified_search_results&m=classified_search_results_classified_clas
sified_detail_XL
2    https://www.immowelt.de/expose/591c45d0-01dd-4baf-8f0b-626cf4b3d692?serp_view=
list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3
DAD08DE8634#ln=classified_search_results&m=classified_search_results_classified_clas
sified_detail_XL
Name: listing_url, dtype: object

In [164... # Sample 11 was not available anymore, while sample 2 was available and it had the
df = df.drop(index=11)
df.loc[2, ["cold_rent", "floor"]] = [4100, 3]

In [165... with pd.option_context('display.max_colwidth', None):
    print(df[df['cold_rent'] < 200].sample(5)['listing_url'])

10740    https://www.immowelt.de/expose/af422439-0ca0
-46dc-9f15-638e20af4eb3?serp_view=list&search=distributionTypes%3DRent%26estateType
s%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D98#ln=classified_search_resu
lts&m=classified_search_results_classified_classified_detail_XL
12966    https://www.immowelt.de/expose/0a01010e-a4ba
-493c-9d08-348ec6167f6d?serp_view=list&search=distributionTypes%3DRent%26estateType
s%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D14#ln=classified_search_resu
lts&m=classified_search_results_classified_classified_detail_XL
9384    https://www.immowelt.de/expose/932e5f77-56e
1-4521-bd03-5859d065861b?serp_view=list&search=distributionTypes%3DRent%26estateType
s%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D73#ln=classified_search_resu
lts&m=classified_search_results_classified_classified_detail_M
2013    https://www.immowelt.de/expose/92d3ddc5-302c-437b-abb0-cc87623a9864?serp_vi
ew=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26location
s%3DAD08DE8634%26page%3D80&feature_priceDropVariantB=true#ln=classified_search_resul
ts&m=classified_search_results_classified_classified_detail_XL
322    https://www.immowelt.de/expose/6243b2cf-1754
-437e-b94e-6a55381f09a1?serp_view=list&search=distributionTypes%3DRent%26estateType
s%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D164#ln=classified_search_res
ults&m=classified_search_results_classified_classified_detail_L
Name: listing_url, dtype: object

In [166... # They were all legit, they explicitly mentioned the price even in the description,
# But somehow there were some duplicates, which need to be removed.
df = df.drop(index=[10740, 1713, 1857])

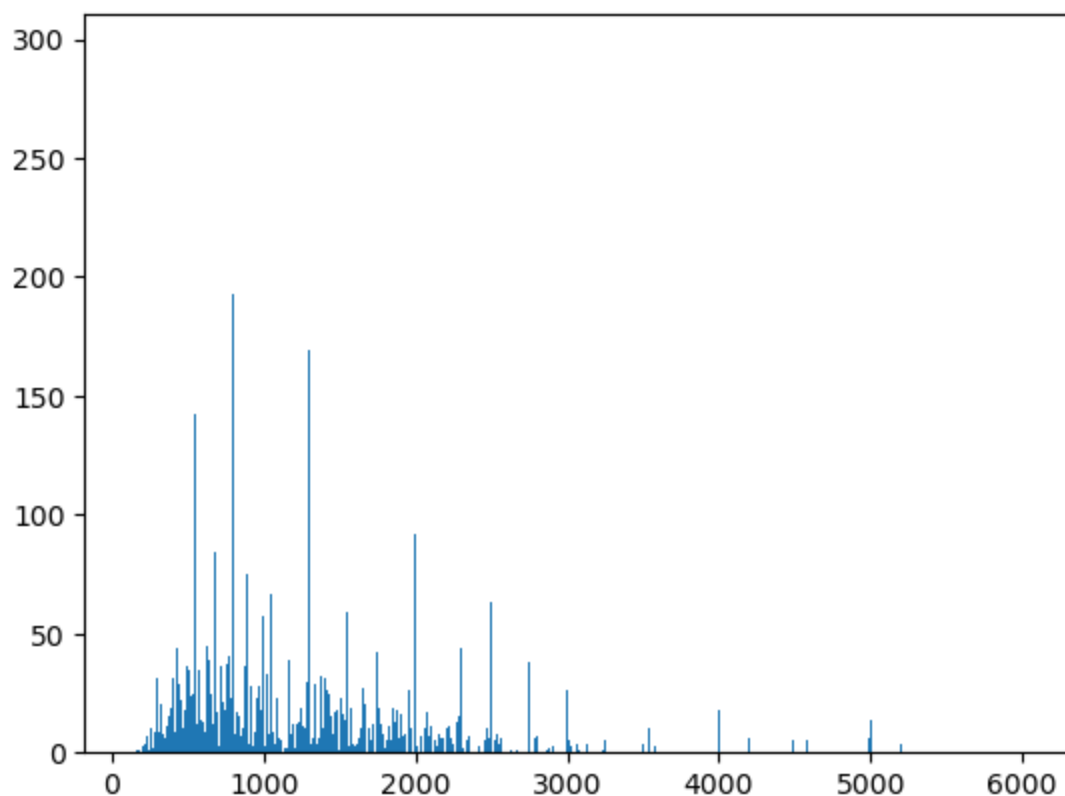
In [167... # We know that 25% of the samples have a cold rent higher than 1642 and the maximum
df["cold_rent"].quantile([0.90, 0.95, 0.975, 0.985, 0.99, 0.995, 0.999])

```

```
Out[167... 0.900    2240.00
           0.950    2700.00
           0.975    3300.00
           0.985    4750.00
           0.990    6061.87
           0.995    7175.00
           0.999   12000.00
Name: cold_rent, dtype: float64
```

```
In [168... # Let's go with the 99% winsorization, since it only means removing 1% of about 160
# Also these samples do not help our model in any way because it does not interest
# just different floors of the same building.
threshold = df["cold_rent"].quantile(0.99)
df = df[df["cold_rent"] <= threshold]
```

```
In [169... plt.hist(df['cold_rent'],bins=2000)
plt.show()
```



```
In [170... df["cold_rent"].quantile([0.90, 0.95, 0.975, 0.985, 0.99, 0.995, 0.999])
```

```
Out[170... 0.900    2200.000
           0.950    2595.000
           0.975    2990.000
           0.985    3300.000
           0.990    3775.000
           0.995    4750.000
           0.999    5418.125
Name: cold_rent, dtype: float64
```

```
In [171... # Now the percentiles are have more fair values and the graph of price distribution  
# to the next feature let's just have one more look at the statistics of the prices  
df["cold_rent"].describe()
```

```
Out[171... count    15626.000000  
mean      1239.583287  
std        743.644768  
min        120.000000  
25%        675.000000  
50%        1100.000000  
75%        1613.000000  
max        6060.000000  
Name: cold_rent, dtype: float64
```

## 2. Handling 'area' feature

```
In [172... df["area"].describe()
```

```
Out[172... count    15626.000000  
mean       78.653923  
std        117.657379  
min         1.000000  
25%         50.500000  
50%         65.800000  
75%         86.800000  
max        5251.000000  
Name: area, dtype: float64
```

```
In [173... # anything below 10 wouldn't make sense  
df[df["area"] < 10]
```

Out[173...

|              | listing_url                                       | cold_rent | area | rooms | floor | free  |
|--------------|---------------------------------------------------|-----------|------|-------|-------|-------|
| <b>12941</b> | https://www.immowelt.de/expose/bf9188d4-e4b2-4... | 2099.0    | 1.0  | 5.0   | 3.0   | immed |
| <b>5550</b>  | https://www.immowelt.de/expose/bf9188d4-e4b2-4... | 2099.0    | 1.0  | 5.0   | 1.0   | immed |
| <b>4604</b>  | https://www.immowelt.de/expose/76343a42-0fc7-4... | 2600.0    | 1.0  | 6.0   | 1.0   | immed |
| <b>69</b>    | https://www.immowelt.de/expose/bf9188d4-e4b2-4... | 2099.0    | 1.0  | 5.0   | 0.0   | immed |
| <b>1457</b>  | https://www.immowelt.de/expose/e29bc640-6917-4... | 418.0     | 1.0  | 3.0   | 0.0   | unk   |

5 rows × 7 columns



In [174...

```
# These are all legitimate samples with legitimate valuse except area. Keeping them
df = df[df["area"] >= 10]
```

Since we decided to keep the luxury listings, we cannot remove anything from the higher tail.

### 3. Handling 'rooms' feature

In [175...

```
df['rooms'].unique()
```

Out[175...

```
array([ 2. ,  1. ,  4. ,  5. ,  3. ,  1.5,  3.5,  8. ,  7. ,  2.5,  6. ,
        5.5,  4.5,  6.5, 10. ])
```

All of them are legitimate values.

### 4. Handling 'floor' feature

In [176...

```
df['floor'].unique()
```

Out[176...

```
array([nan, 21., 20., 18., 17., 16., 15., 14., 13., 12., 11., 10.,  9.,
        8.,  7.,  6.,  5.,  4.,  3.,  2.,  1.,  0., -1.])
```

All of them are legitimate values.

### 5. Handling 'property\_condition' feature

In [177...

```
df['property_condition'].value_counts()
```

```
Out[177... property_condition
Unknown          9284
New / First Occupancy  1796
Like New         1609
Well Maintained   1453
Renovated        1306
Old Building      56
Partially Renovated  44
Needs Renovation  38
Other            35
Name: count, dtype: int64
```

Every category has its own meaning, so removing any of them would cause information loss. Removing the 'Other' category would cause losing valuable samples, so there is nothing we can adjust in this feature.

## 6. Handling 'year\_built' feature

```
In [178... df['year_built'].describe()
```

```
Out[178... count          7688.0
mean         1996.926119
std           43.860206
min           1765.0
25%           1987.5
50%           2020.0
75%           2024.0
max           2026.0
Name: year_built, dtype: Float64
```

All of them are legitimate values.

Now let's just remove the features that aren't needed and the dataset will be completely cleaned.

```
In [179... df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 15621 entries, 22653 to 3
Data columns (total 27 columns):
#   Column                Non-Null Count  Dtype
---  -
0   listing_url            15621 non-null  object
1   cold_rent              15621 non-null  float64
2   area                  15621 non-null  float64
3   rooms                 15621 non-null  float64
4   floor                 13673 non-null  float64
5   free_from             15621 non-null  object
6   location              15621 non-null  object
7   has_balkon            15621 non-null  boolean
8   has_terrasse          15621 non-null  boolean
9   has_garten            15621 non-null  boolean
10  elevator              15621 non-null  boolean
11  parking               15621 non-null  boolean
12  has_basement          15621 non-null  boolean
13  is_barrier-free       15621 non-null  boolean
14  has_built-in_kitchen  15621 non-null  boolean
15  has_bathtub           15621 non-null  boolean
16  has_shower            15621 non-null  boolean
17  flooring_type         15621 non-null  object
18  energy_source         15621 non-null  object
19  heating_type          15621 non-null  object
20  property_condition    15621 non-null  object
21  year_built            7688 non-null   Int64
22  schufa_check          15621 non-null  bool
23  scraped_at            15621 non-null  datetime64[ns]
24  location_clean        15621 non-null  object
25  zip_code              15621 non-null  object
26  district              15621 non-null  object
dtypes: Int64(1), bool(1), boolean(10), datetime64[ns](1), float64(4), object(10)
memory usage: 2.4+ MB
```

## Part 5: Privacy & Re-identification Risk Assessment (k-Anonymity Check)

```
In [180... def k_anonymity_report(df, cols, label=None):
    group_sizes = df.groupby(cols, dropna=False).size()
    n_singletons = (group_sizes == 1).sum()

    print(f"--- {label or cols} ---")
    print(f"Rows: {len(df)}, Groups: {len(group_sizes)}")
    print(f"Minimum group size (worst-case k): {group_sizes.min()}")
    print(f"Groups with k=1 (fully unique row): {n_singletons} ({n_singletons/len(df)}%)")
    print(f"Groups with k<=5: {(group_sizes <= 5).sum()}")
    print()
    return group_sizes
```

```
In [181... sets_to_test = {
    "district only": ["district"],
    "district + rooms": ["district", "rooms"],
    "district + rooms + floor": ["district", "rooms", "floor"],
```

```

    "district + rooms + floor + year_built": ["district", "rooms", "floor", "year_b
}

results = {label: k_anonymity_report(df, cols, label) for label, cols in sets_to_te

```

--- district only ---

Rows: 15621, Groups: 53

Minimum group size (worst-case k): 24

Groups with k=1 (fully unique row): 0 (0.00% of rows)

Groups with k<=5: 0

--- district + rooms ---

Rows: 15621, Groups: 378

Minimum group size (worst-case k): 1

Groups with k=1 (fully unique row): 40 (0.26% of rows)

Groups with k<=5: 110

--- district + rooms + floor ---

Rows: 15621, Groups: 2500

Minimum group size (worst-case k): 1

Groups with k=1 (fully unique row): 912 (5.84% of rows)

Groups with k<=5: 1833

--- district + rooms + floor + year\_built ---

Rows: 15621, Groups: 6208

Minimum group size (worst-case k): 1

Groups with k=1 (fully unique row): 4084 (26.14% of rows)

Groups with k<=5: 5694

In [182...

```

df["area_bin"] = (df["area"] // 10 * 10).astype(int)

k_anonymity_report(df, ["district", "rooms", "area_bin"], "district + rooms + area_
k_anonymity_report(df, ["district", "rooms", "floor", "area_bin", "year_built"], "w

```

--- district + rooms + area\_bin ---

Rows: 15621, Groups: 1344

Minimum group size (worst-case k): 1

Groups with k=1 (fully unique row): 258 (1.65% of rows)

Groups with k<=5: 710

--- worst case ---

Rows: 15621, Groups: 9115

Minimum group size (worst-case k): 1

Groups with k=1 (fully unique row): 6620 (42.38% of rows)

Groups with k<=5: 8765



```
Out[182...] district    rooms  floor  area_bin  year_built
Alt-Treptow  1.0    0.0    20        2021        1
              2022        1
              30        1950        1
              <NA>        2
              40        <NA>        1
              ..
Zehlendorf   5.0    4.0    800        1978        1
              5.0    140        2009        1
              6.0    170        2016        1
              NaN    170        2016        1
              400        1930        1

Length: 9115, dtype: int64
```

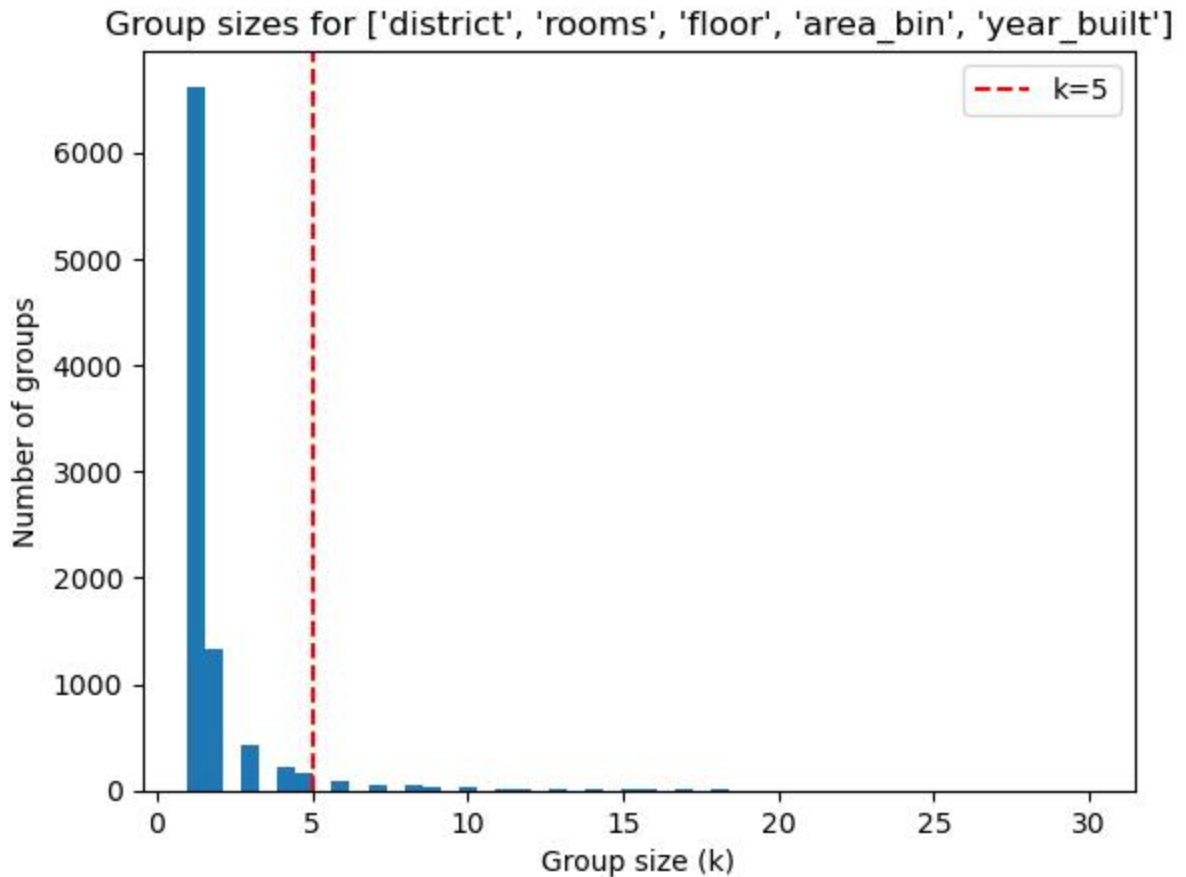
```
In [183...] worst_cols = ["district", "rooms", "floor", "area_bin", "year_built"]
sizes = df.groupby(worst_cols, dropna=False).size().rename("group_size")

df_sized = df.join(sizes, on=worst_cols)
unique_rows = df_sized[df_sized["group_size"] == 1]
print(f"{len(unique_rows)} rows are unique on {worst_cols}")
unique_rows[worst_cols + ["cold_rent", "area"]].head(10)
```

6620 rows are unique on ['district', 'rooms', 'floor', 'area\_bin', 'year\_built']

```
Out[183...]      district  rooms  floor  area_bin  year_built  cold_rent  area
22653    Prenzlauer Berg    2.0  NaN      50      1910    1650.00  54.0
22652           Wedding    1.0  NaN      20      1969     584.00  29.2
22642           Mitte    5.0  NaN     200      <NA>    4750.00 203.0
22637    Charlottenburg    3.0  NaN     140     1899    3270.00 148.0
22622       Bohnsdorf    3.0  NaN      90     2021    1640.49  93.0
22612       Schöneberg    1.0  NaN      50     2019    1218.48  50.8
22604  Französisch Buchholz    4.0  NaN      80     2022    1958.00  89.0
22579           Tegel    3.0  NaN      80     1955    1550.00  83.0
22551       Friedenau    3.0  NaN     100     1905    1881.00 104.5
22534       Lichtenberg    1.5  NaN      50     1930     804.00  50.3
```

```
In [184...] plt.hist(sizes, bins=50)
plt.axvline(5, color="red", linestyle="--", label="k=5")
plt.xlabel("Group size (k)")
plt.ylabel("Number of groups")
plt.title(f"Group sizes for {worst_cols}")
plt.legend()
plt.show()
```



Generalizing location down to district alone is effectively safe on its own: the smallest district contains 24 listings, and no district value isolates a row by itself. That safety margin collapses quickly once ordinary structural columns are added on top. Adding rooms already produces 40 fully unique rows (0.26%), and adding floor pushes this to 912 (5.84%) — notably, many of these are unique specifically because a missing floor value paired with a rare rooms/district combination becomes its own distinguishing signature, not because of numeric precision. `year_built` is the single biggest driver of uniqueness: layering it on top of district+rooms+floor raises the singleton rate to 26.14%, since exact construction year is high-cardinality and only present for roughly half the dataset. In the realistic worst case — district, rooms, floor, a 10m<sup>2</sup> area bucket, and `year_built` together — 42.38% of all rows are uniquely identifiable, meaning nearly half the dataset could in principle be traced back to one specific real listing using nothing but ordinary attributes, with no need for location, `zip_code`, or `listing_url`. Dropping location and `listing_url` was a necessary step but was clearly not sufficient on its own to prevent re-identification.

If we were to share or publish the dataset, we have to generalize the two biggest offenders before release: bucket `year_built` into ranges instead of exact years (e.g. decades), and group floor into a few ranges (e.g. ground / low / mid / high) rather than exact floor number. For our model training we are going to use the current dataset.

## Part 6: Saving the dataset

```
In [185... df = df.drop(columns=["location", "location_clean", "scraped_at", "area_bin"])
```

```
In [186... df.to_csv("../datasets/rent_listings.csv", encoding="utf-8-sig", index=False)
```